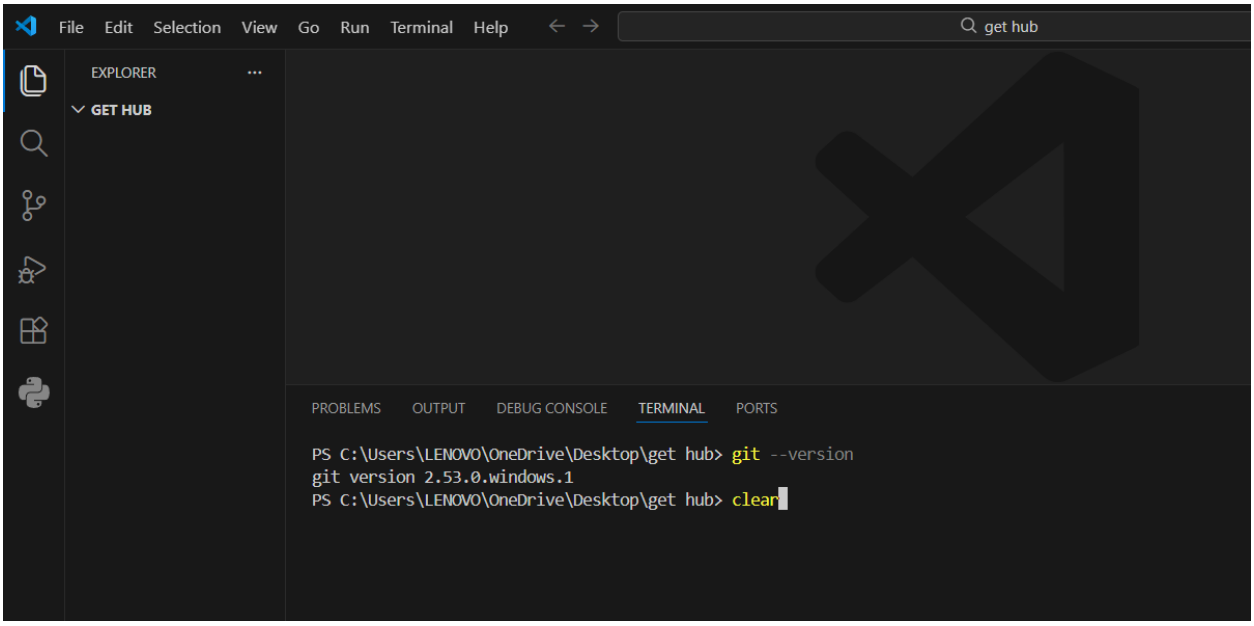
# 📘 Git & GitHub — Complete Beginner Notes (Till Now)

---

## 1️⃣ What is a git? (Core Idea)




**Git = Version Control System**

Simple meaning:

👉 Git is a system that saves the history of a project.

Just like:

- A writer → keeps a diary
- A scientist → keeps a lab notebook
- A developer → uses Git

Git remembers every change.

---

What problem does Git solve?

Without Git ❌

- file delete → gone forever

- Mistakes → Difficult to recover
- Old versions → Unavailable

With Git ✅

- Revert to old versions
- Change tracking
- Safe experimentation

Git = **Project Time Machine** ⏳

---

## 2️⃣ Git vs GitHub

| **Git** | **GitHub** |
|---|---|
| Software | Website |
| Runs on a computer | Runs on the Internet |
| Saves history | Stores projects online |
| Offline work | Online sharing |

👉 Git = engine
👉 GitHub = cloud storage

---

## 3️⃣ What is the role of VS Code?

VS Code = Editor (workspace)

- You edit files.
- You manage projects.
- You run Git commands.

---

## 4️⃣ Terminal Basics (Navigation)

Terminal = Communicating directly with a computer.

## Important Commands

📁 Folder banana

Bash
```bash
mkdir folder-name
```

📂 Folder ke andar jana

Bash
```bash
cd folder-name
```

⬅️ Folder ke bahar aana

Bash
```bash
cd ..
```

📋 Folder list dekhna

Bash
```bash
ls
```

📍 Current location

Bash
```bash
pwd
```

```
LENOVO@DESKTOP-4U39T15 MINGW64 ~
$ mkdir papu

LENOVO@DESKTOP-4U39T15 MINGW64 ~
$ cd papu

LENOVO@DESKTOP-4U39T15 MINGW64 ~/papu
$ ls

LENOVO@DESKTOP-4U39T15 MINGW64 ~/papu
$ pwd
/c/Users/LENOVO/papu

LENOVO@DESKTOP-4U39T15 MINGW64 ~/papu
$
```

**Mental Model**

**Computer = City 🏙️**

- **folder = building**
- **cd = walking**
- **ls = view map**

---

## 5️⃣ Git Setup

**It is essential to set an identity.**

```bash
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

## 6️⃣ Git Repository kya hota hai?

**Repository (repo) = Git project folder.**

Create karne ke liye:

```bash
git init
```

Ye folder ko smart bana deta hai.

Hidden folder banega:

```code
.git
```

Ye Git ka brain hai 🧠.

## 7️⃣ File Change Workflow (MOST IMPORTANT)

**When you edit a file…**

STEP 1 — Normal save

Code

```
Ctrl + S
```

STEP 2 — Status check

Bash

```bash
git status
```

STEP 3 — Changes add

Bash

```bash
git add .
```

STEP 4 — History save

Bash

```bash
git commit -m "message"
```

---

Golden Formula ⭐

Bash

```bash
git add .
git commit -m "what changed"
```

**What is a commit?**

**Commit = A snapshot of the project 📸**

**Every commit is a saved version.**

**Example:**

Code

```
Commit 1 → start
Commit 2 → feature added
Commit 3 → bug fixed
```

**9️⃣ Git Status Colors (VS Code)**

| Color | Meaning |
| --- | --- |
| Red | file changed |
| Green | ready to commit |

---

**What is Git Clone?**

**Project download from GitHub.**

```bash
git clone REPO-LINK
```

**Git automatically:**

- ✅ **making folder**
- ✅ **files download**

---

**Clone Error (Whatever came to you)**

```
destination path already exists
```

Meaning:

👉 same naam ka folder already hai.

Fix:

```bash
cd ..
git clone link
```

**1️⃣ Folder Naming Rule**

**The terminal treats a space as a separate word.**

Wrong ❌

Code

```
mkdir my project
```

Correct ✅

Code

```
mkdir "my project"
```

Best ✅

Code

```
mkdir my-project
```

## 1️⃣2️⃣ Basic Git Flow (Full Picture)

Code

```
Create folder
↓
git init
↓
Create file
↓
git add .
↓
git commit
↓
Repeat changes
```

## 1️⃣3️⃣ Real Developer Workflow

**Developer daily:**

**edit file**
**→ save**
**→ git add**
**→ git commit**

**Repeat many times.**

---

**14 Why Git Important for AI / Coding Career**

**Because:**

- **portfolio building**
- **made projects proof**
- **internships examine GitHub profile**
- **collaboration possible**

**GitHub = developer resume.**

---

**Ultimate Understanding**

**Git is not actually coding tool.**

**Git = thinking tool.**

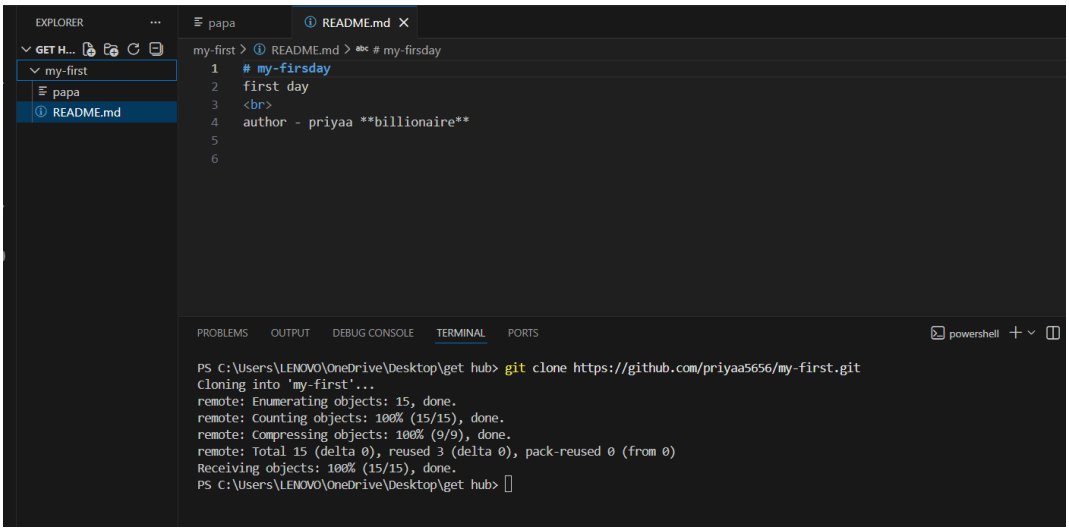
**this allows:**

- **fear-free learning**
- **experimentation**
- **mistake recovery**

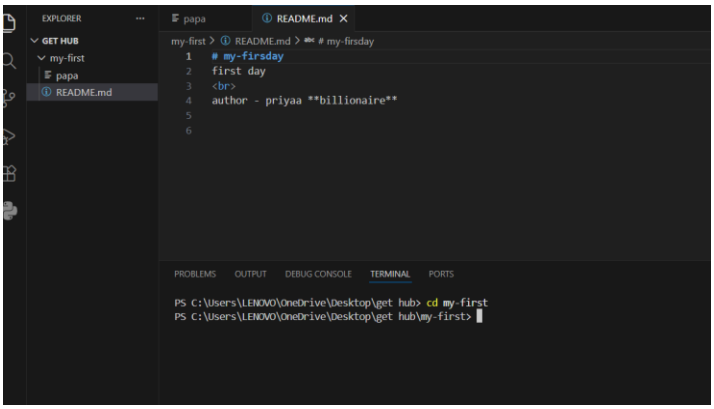
## commands :

### (a) git clone -----

This is used to clone a GitHub repository into your computer. remote means is github and local means laptop or pc. All the files that were on GitHub will be cloned here.

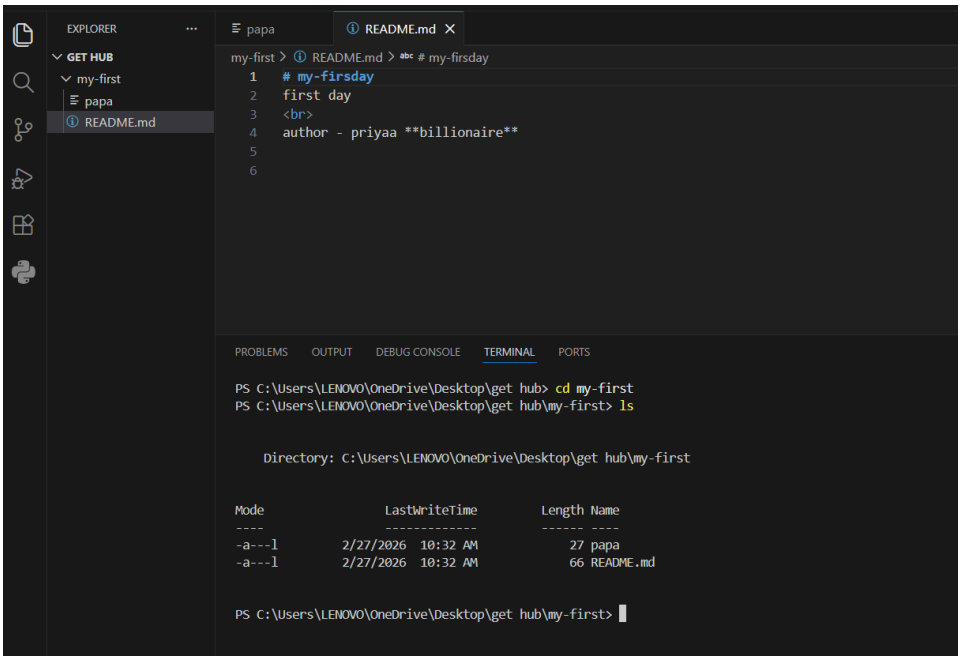



## (b) change the directory {folder}



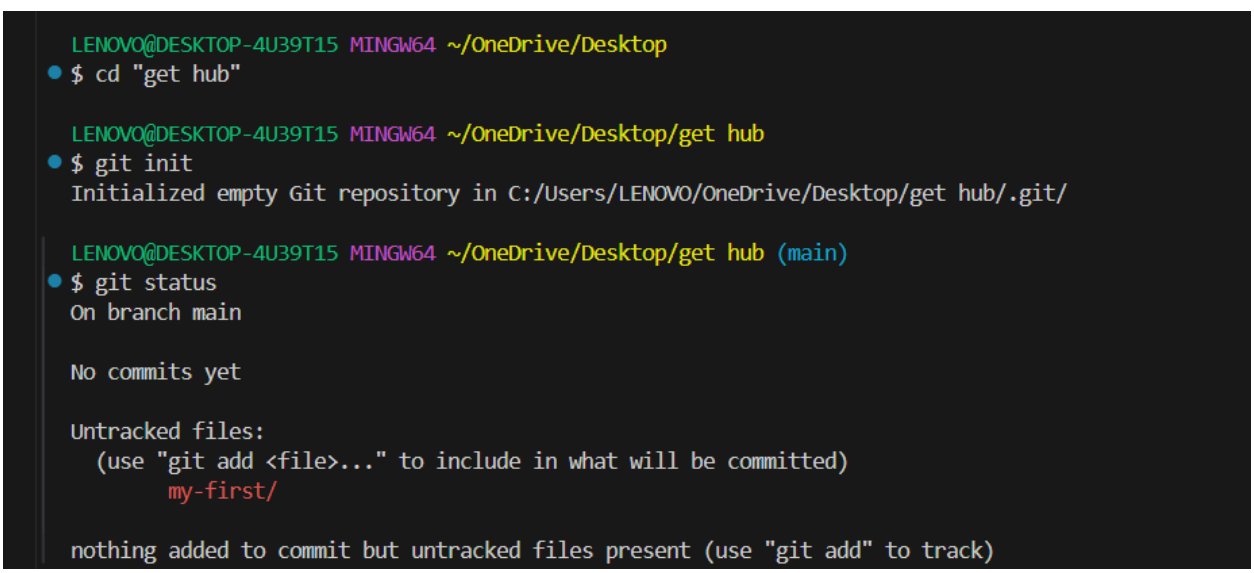

## © ls

To see how many file in our folder use Ls (list file).




## (c) ls -a

in terminal we have two type of file one is normal file and 2nd is hidden file. If we want to see hidden file use ls -a.

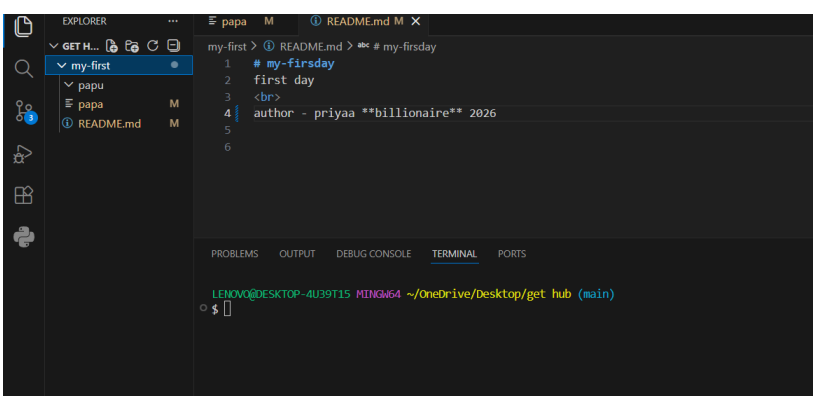


```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub (main)
$ git add .
warning: adding embedded git repository: my-first
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> my-first
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached my-first
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub (main)
$ git commit -m "first commit"
[main (root-commit) f80fb39] first commit
 1 file changed, 1 insertion(+)
 create mode 160000 my-first

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub (main)
$ git status
On branch main
nothing to commit, working tree clean
```


```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub (main)
$ git commit -m "first commit"
[main (root-commit) f80fb39] first commit
 1 file changed, 1 insertion(+)
 create mode 160000 my-first

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub (main)
$ git status
On branch main
nothing to commit, working tree clean
```

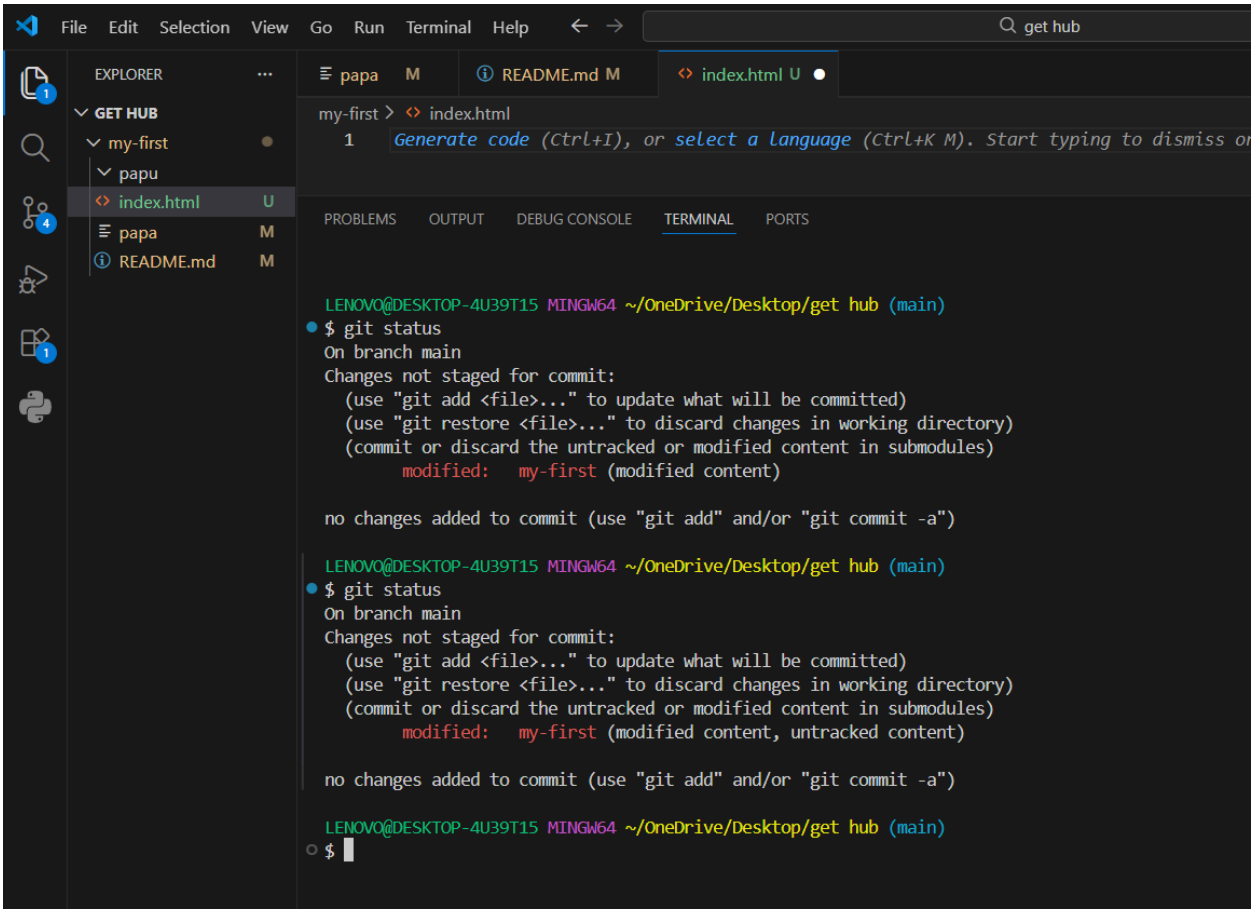

```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub (main)
$ ls
my-first/

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub (main)
$ ls -a
./  ../  .git/  my-first/
```

## Save shortcut---- ctrl+s



## Git status

The 'M' appearing next to the file indicates that changes have been made in the VS Editor that differ from the version on GitHub and have not yet been committed.




There is so much status type;-

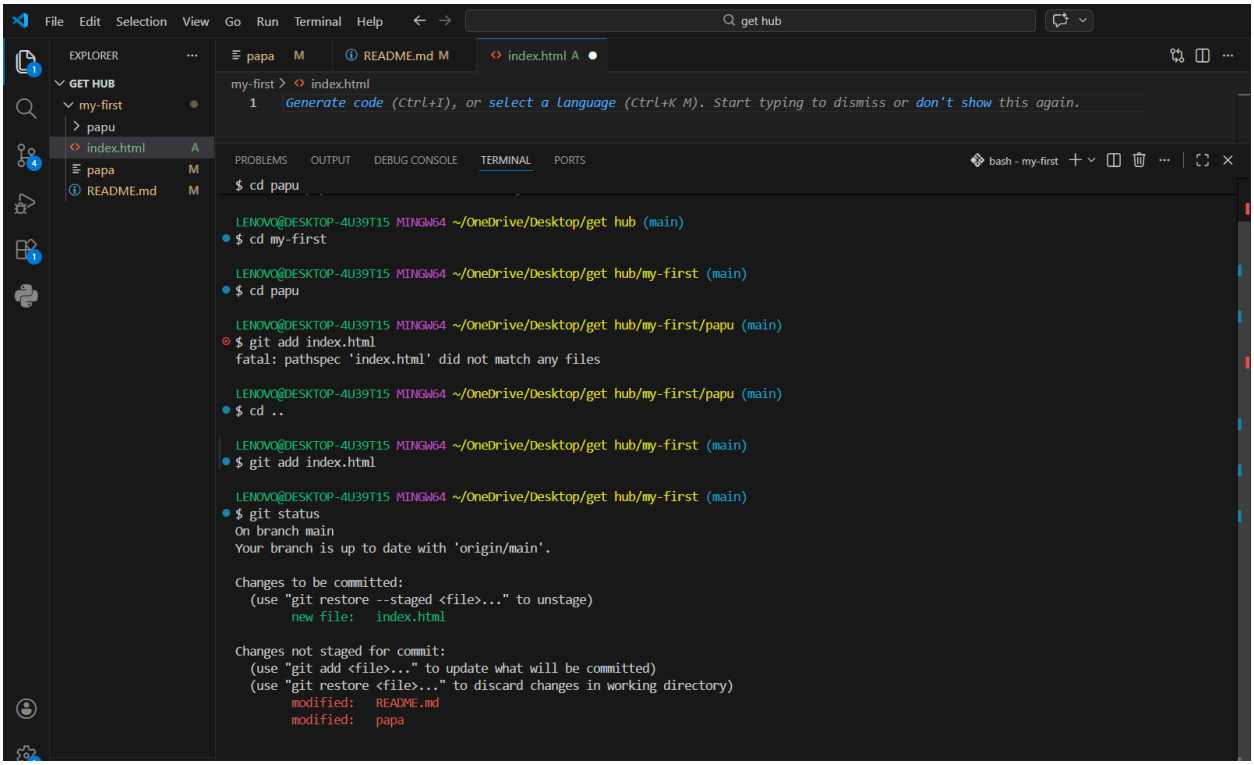
1. untracked----- there is new files that git doesn't yet track.git think these are new file because you don't commit these.
2. Modified------ there is file which is modified or changed
3. Staged --------- this file is ready to be committed
4. Unmodified--- unchanged

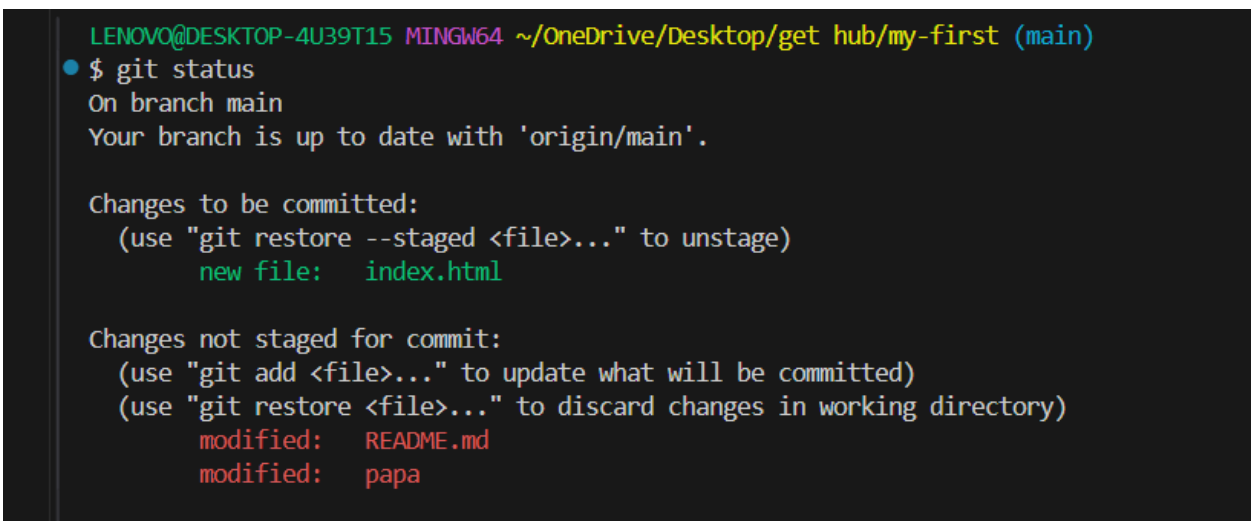
For commit a file we have process -----

new file (untracked)/ changed file(modified file) → add(staged)→ commit (unchanged)

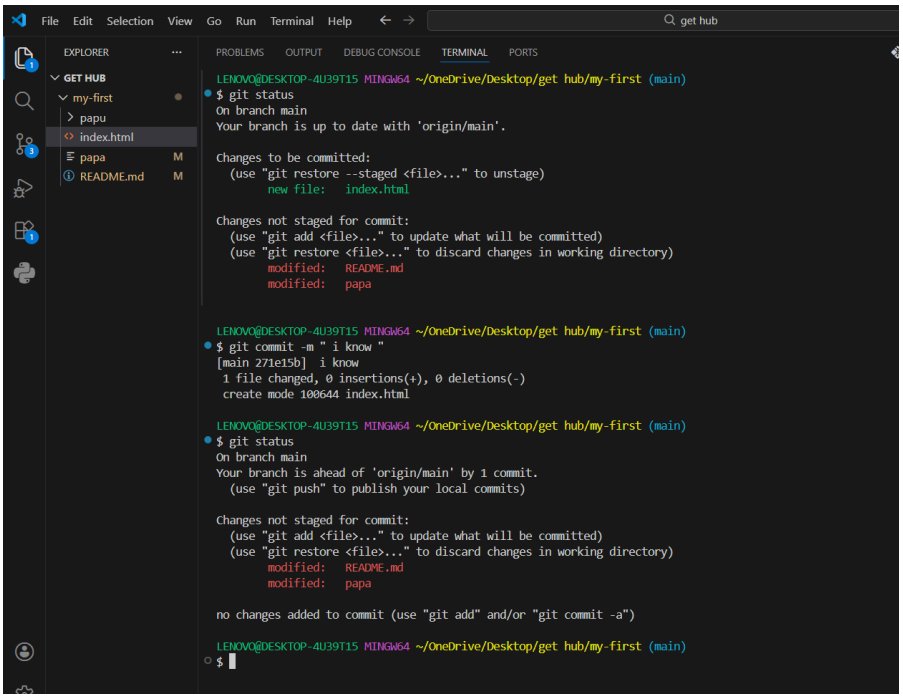
# Add & commit →

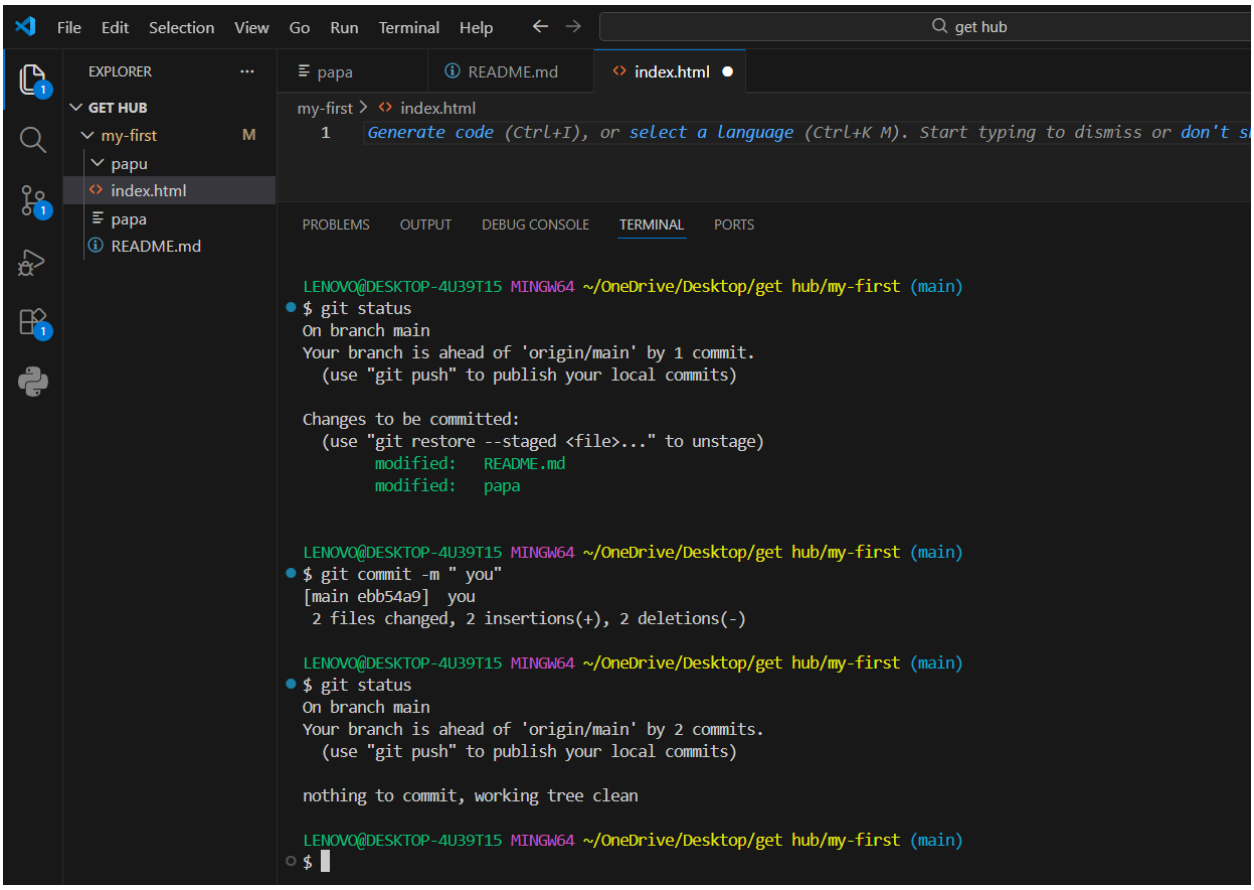
Add – adds new or changed files in your working directory to git staging area.



Commit→ it is the record of change.

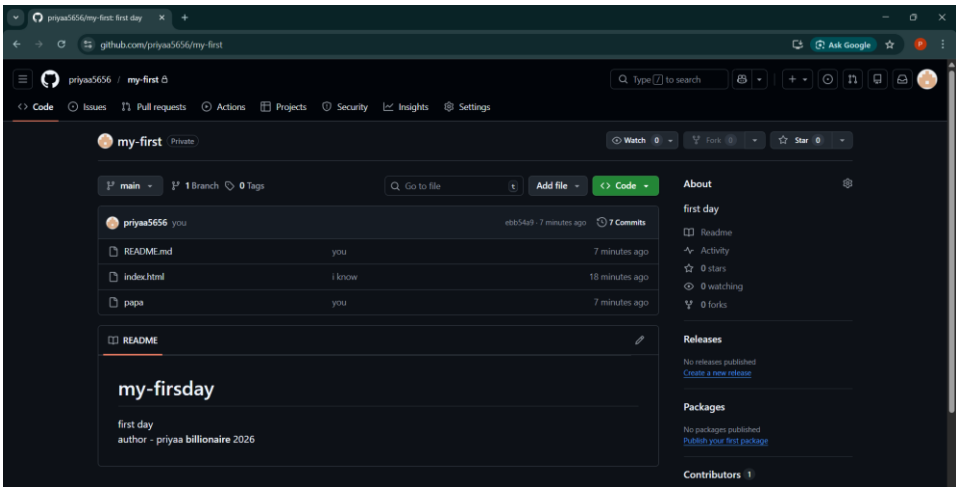


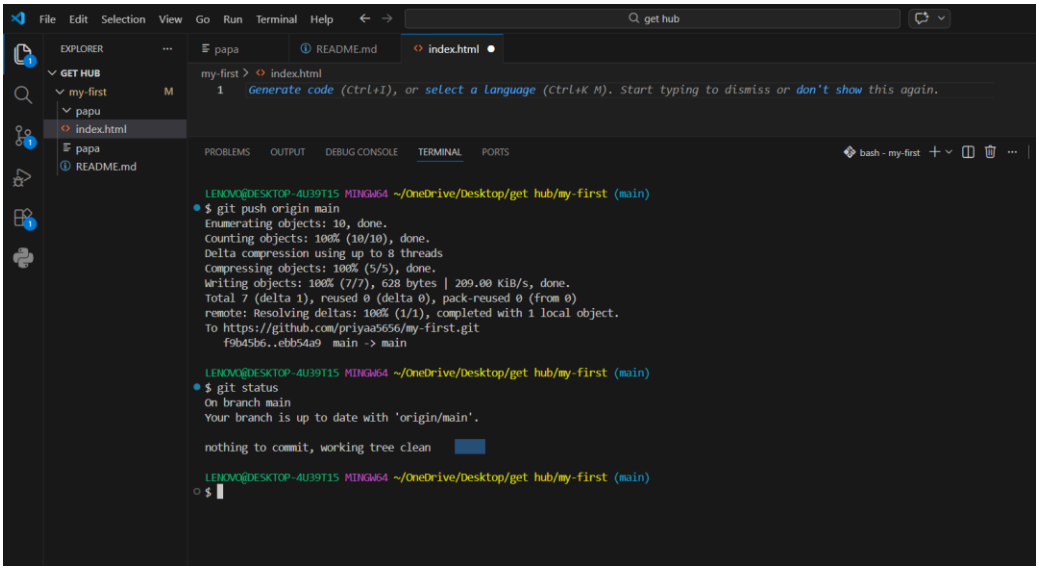



# Push command→

There I see a line ( your branch is ahead of main by 1 commit.) means your computer work is ahead of github repo. These are not same. Use push command. Upload your computer repo content to github repo.

```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/my-first (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
        modified:   papa

no changes added to commit (use "git add" and/or "git commit -a")
```



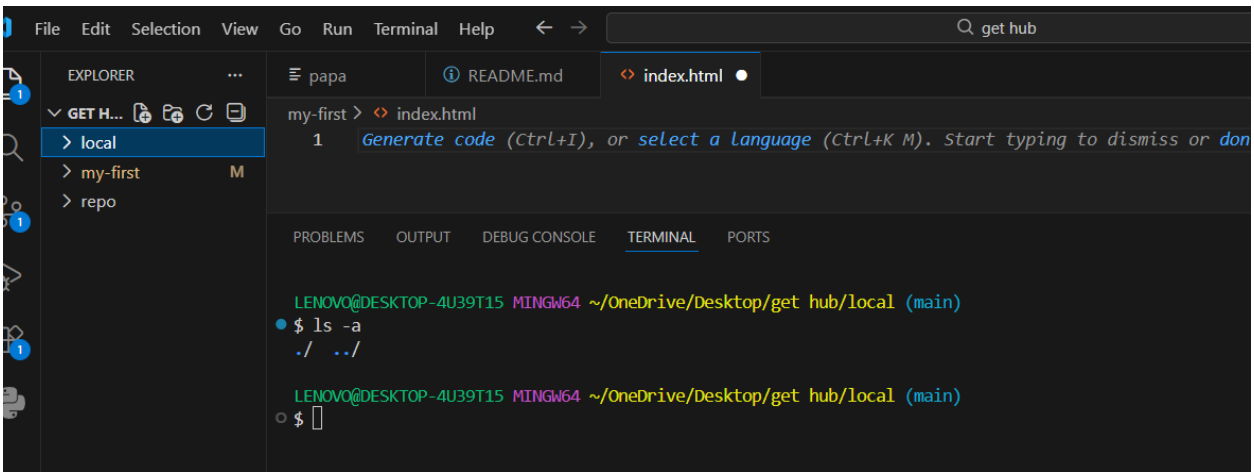



Git push origin main ---- git push is normal command and in remote repo a origin repo (where we cloning) is our remote github and we push our vs content to branch ki main repo.

# Git init command→

Used to create a new repo. If I have new project to start so in this project we want to use git for use git write a init

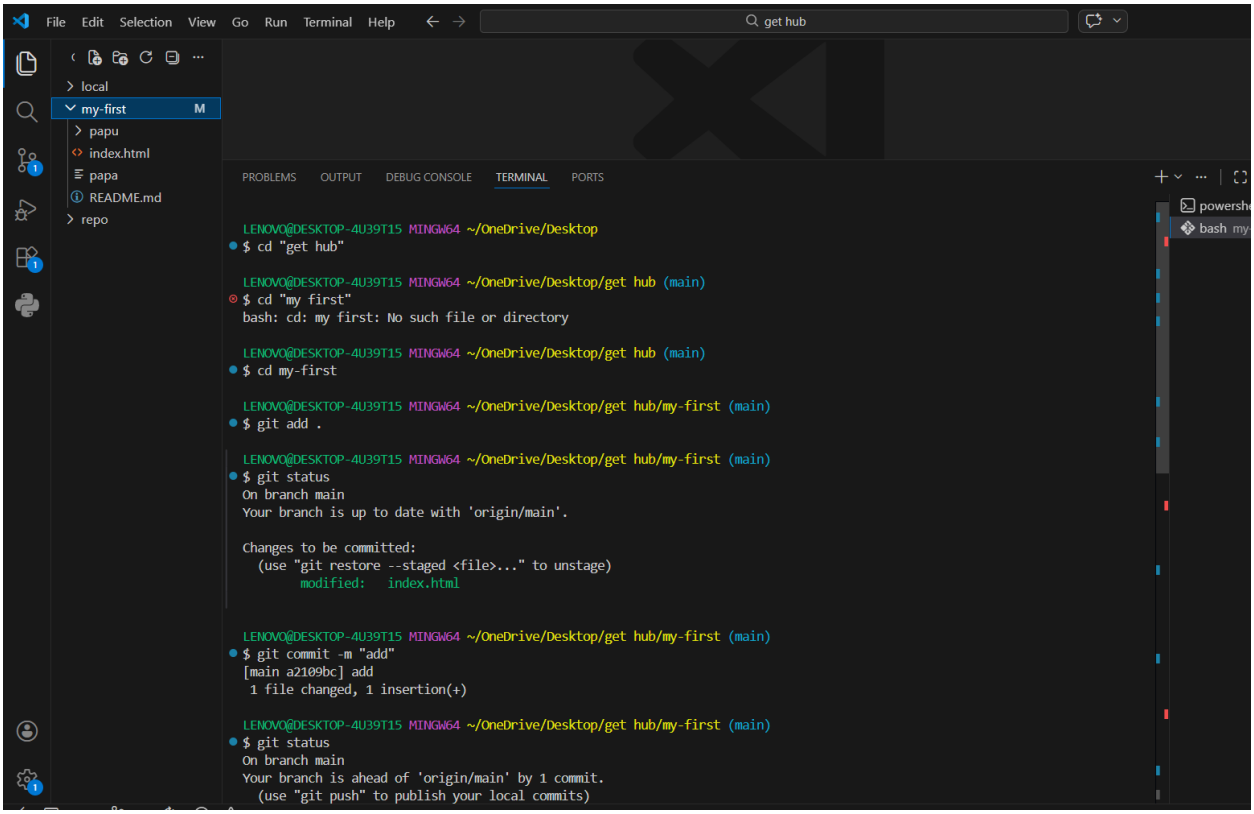
```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/my-first (main)
$ cd ..

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub (main)
$ mkdir local repo

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub (main)
$ cd local

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$
```




We see hidden file with us of ls-a Because there is not .git folder it means this is not git repo. Now we use git init….

```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git init
Initialized empty Git repository in C:/Users/LENOVO/OneDrive/Desktop/get hub/local/.git/

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ ls -a
./  ../  .git/
```

```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html
        style.css

nothing added to commit but untracked files present (use "git add" to track)

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git add .

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git commit -m "add initial files"
[main (root-commit) 798f02c] add initial files
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 index.html
 create mode 100644 style.css

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
```



```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/my-first (main)
$ git status
  (use "git push" to publish your local commits)

nothing to commit, working tree clean

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/my-first (main)
$ git push main origin
fatal: 'main' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/my-first (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 307 bytes | 307.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/priyaa5656/my-first.git
   ebb54a9..a2109bc  main -> main

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/my-first (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

## Git remote add origin <- link->

**You have now reached an interesting point in Git—connecting the local universe with the remote universe. It feels a bit like science fiction: one planet is your laptop, and the other is a server on the internet. Git creates a wormhole through which your code travels. 🚀 The basic idea behind the command you are about to write is this: "Send the project currently on my computer to GitHub. First , let's add the remote link. Here, 'origin' is just a name. It is a convention—people typically refer to the remote repository as 'origin'. You can choose to give it a different name if you wish, but 99% of developers stick with 'origin'.**

****i make a project on my local system now I want this project is must on my github account . I want to add new git hub repo name origin (change or what you want) on github. ****

**git remote add origin <repo-link>**

**Example:**

**git remote add origin https://github.com/username/project.git**

## Git remote-v :

Now check whether the connection has been established or not. (to verify remote)….

## Git branch:

The terminal will show you something like this: This means the local repository and the GitHub repository are now connected.

origin  https://github.com/username/project.git (fetch)

origin  https://github.com/username/project.git (push)

## Git branch-M main

Now, name the branch `main` (the modern Git default). If you want change so go ahead.

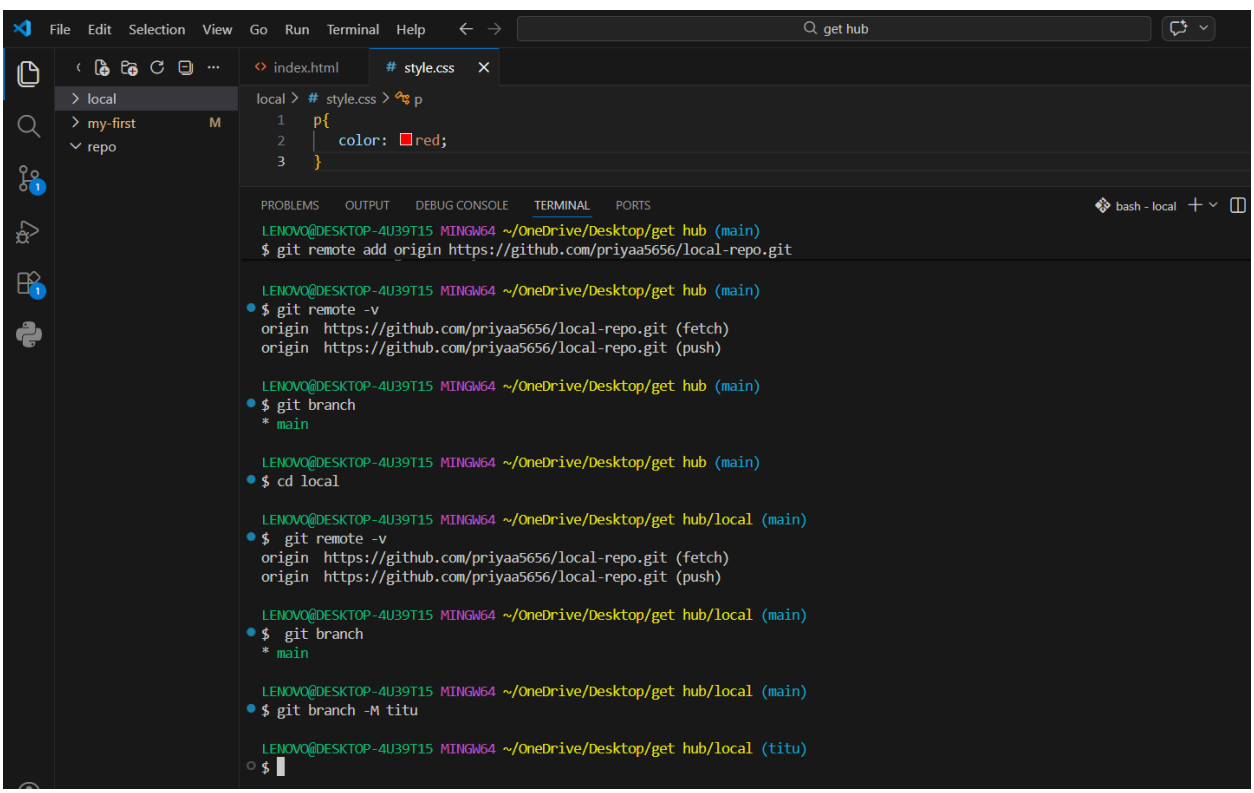
## Git push origin main:

And finally, push the code to GitHub:

git push -u origin main. The `-u` flag stands for setting the upstream. In simple terms:

Git will remember that for future pushes, it should send the changes to this specific remote and branch. Subsequently, the workflow becomes even simpler:

**Means→ if you want to work so long on this project and you don't want to write (git push origin main ) again and again so write git push -u origin main. And now system set this on main .**



File Edit Selection View Go Run Terminal Help

get hub

local
index.html
style.css
my-first M
repo

index.html

local > index.html

```
1   i am a perfect person and happy person
```

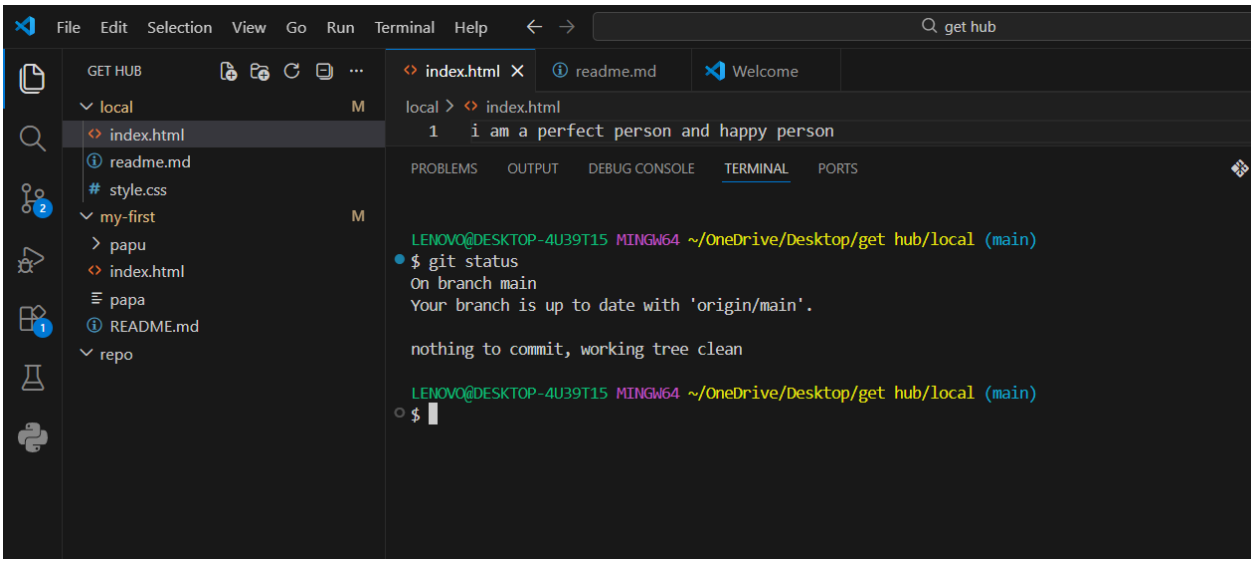
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

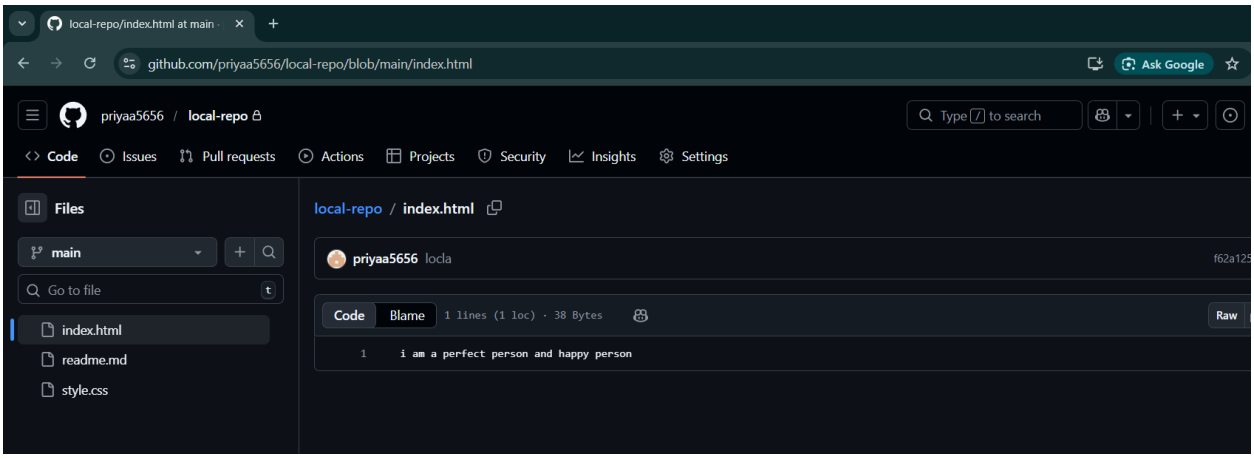
bash - local

```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git branch -M titu

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (titu)
$ git branch -M main

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 766 bytes | 383.00 KiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/priyaa5656/local-repo.git
 * [new branch]      main -> main

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git push
fatal: The current branch main has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin main

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.


LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git push -u origan main
fatal: 'origan' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
```

```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git push -u origan main
fatal: 'origan' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git push
Everything up-to-date
```

**We make new file on git and push to github….**




# Work flow-→

**When we work on local git then our workflow is→ first we make files on git hub (git hub repo). Now we clone it. Then we make changes and then add and commit changes. and finaly we push changes**

**(git hub repo)→ clone → make changes → add → commit → push**

# Git branches➔

The concept of branching in Git is a bit magical. Imagine you are writing a book. One version is the original story. But then you think—"What if the hero heads off into space right here?" So, you start an alternate storyline. In Git, this alternate timeline is called a *branch*. 🌿

Branch = A parallel copy of a project's development history.

The default branch is usually: main

This is your main timeline.

Example--→

```
1 → 2 → 3 → 4 ↘
        ↘       ↘
          5 → 6 → 7
```

**or**

```
main:    ●—●—●—●
                \
feature:          ●—●—●
```

**The circles or number in the diagram actually represent commits. A commit is essentially a saved snapshot of the project.**

**1 → 2 → 3 → 4  This constitutes the main timeline.**

**Now, observe that a branch diverges from the center:**

```
1 → 2 → 3 → 4 ↘
        ↘       ↘
          5 → 6 → 7
```

**This implies that, following Commit 2, a developer tested a new idea. Consequently, a new branch was created.**

## Branch commands-→

## Git branch→

To view branches -- > This will display all the branches currently present in the project.

## Git branch-M main →

To rename a branch. This renames the current branch to 'main'.

## Git checkout <- branch name->

To switch or go to a specific branch.

Example:

git checkout feature

You are now working within the 'feature' branch.

## Git checkout  -b <- new branch name->

4. To create a new branch and switch to it

Example:

git checkout -b login-feature

A new branch has been created, and you have switched to it.

## Git branch  -d  <-  branch name->

**To delete a branch. If we want to delete feature 1 branch so we have to go another branch and then delete feature 1 branch otherwise if you delete this branch , the branch is not deleted.**

**Example:**

**git branch -d login-feature**

**The branch will be deleted.**

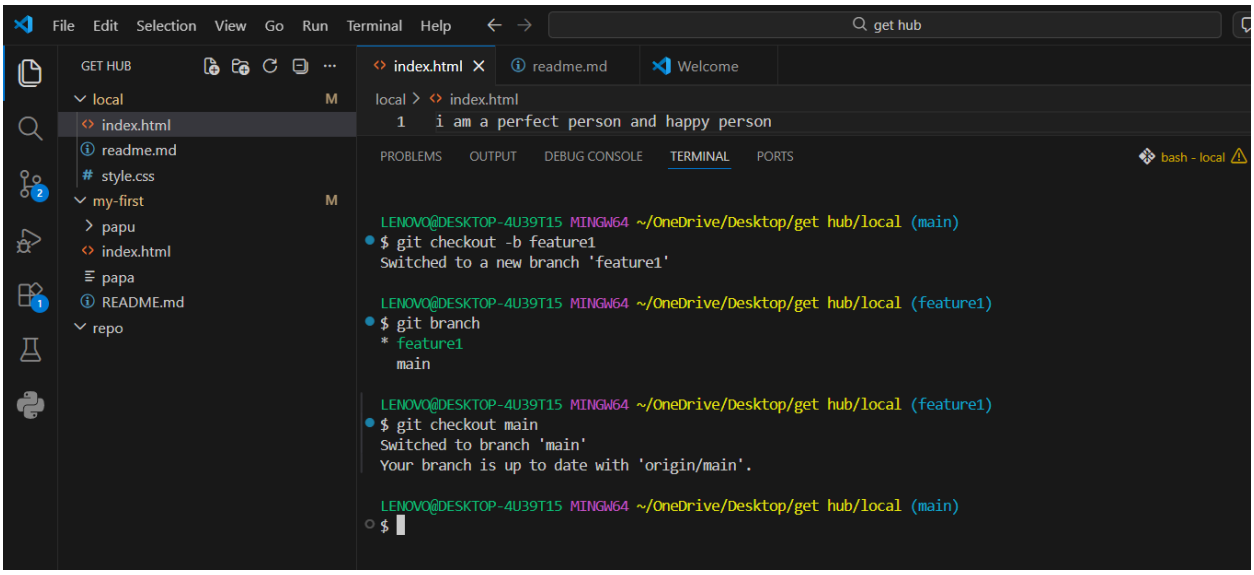
**Simple Memory Trick:**

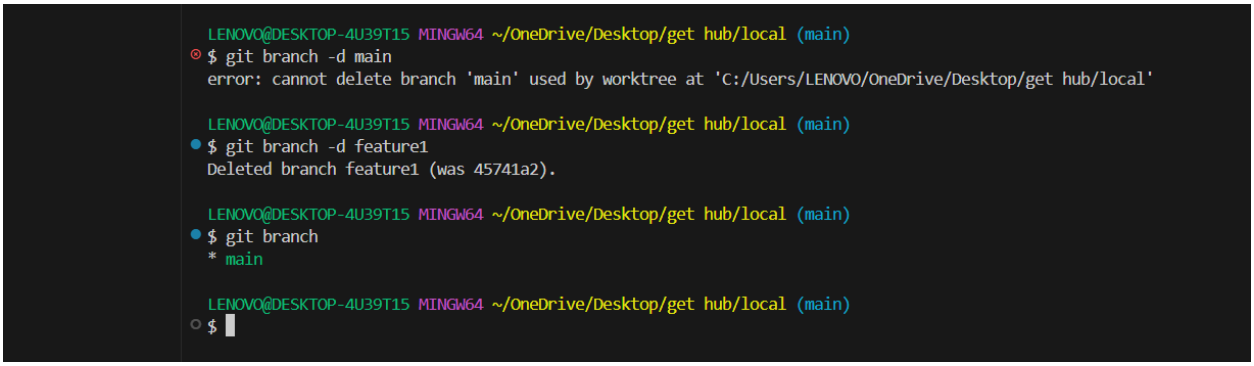
**git branch → View branches**

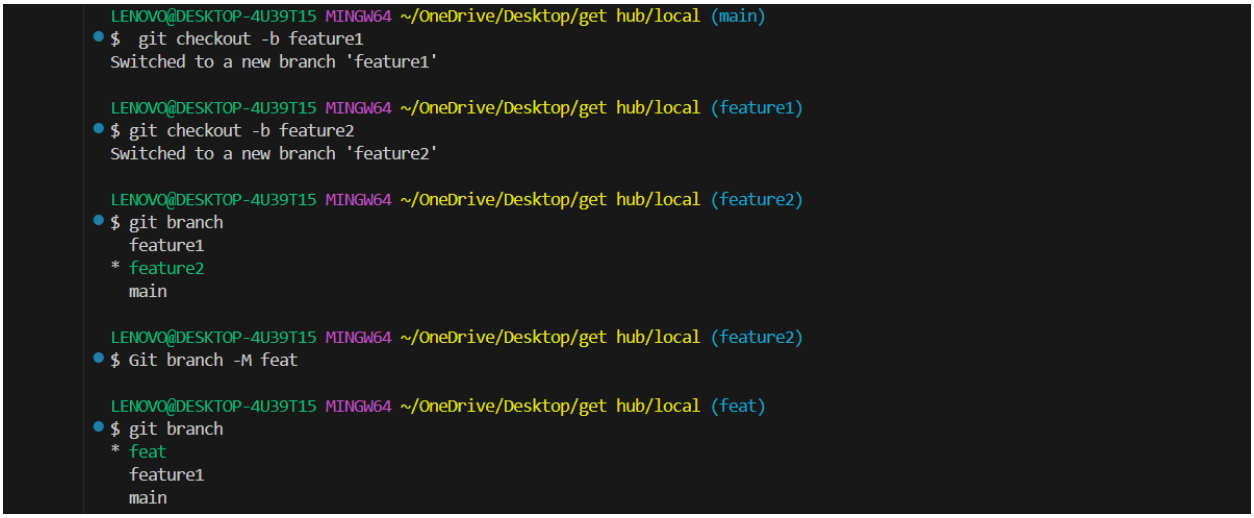
**git checkout → Switch branches**

**git checkout -b → Create a new branch**

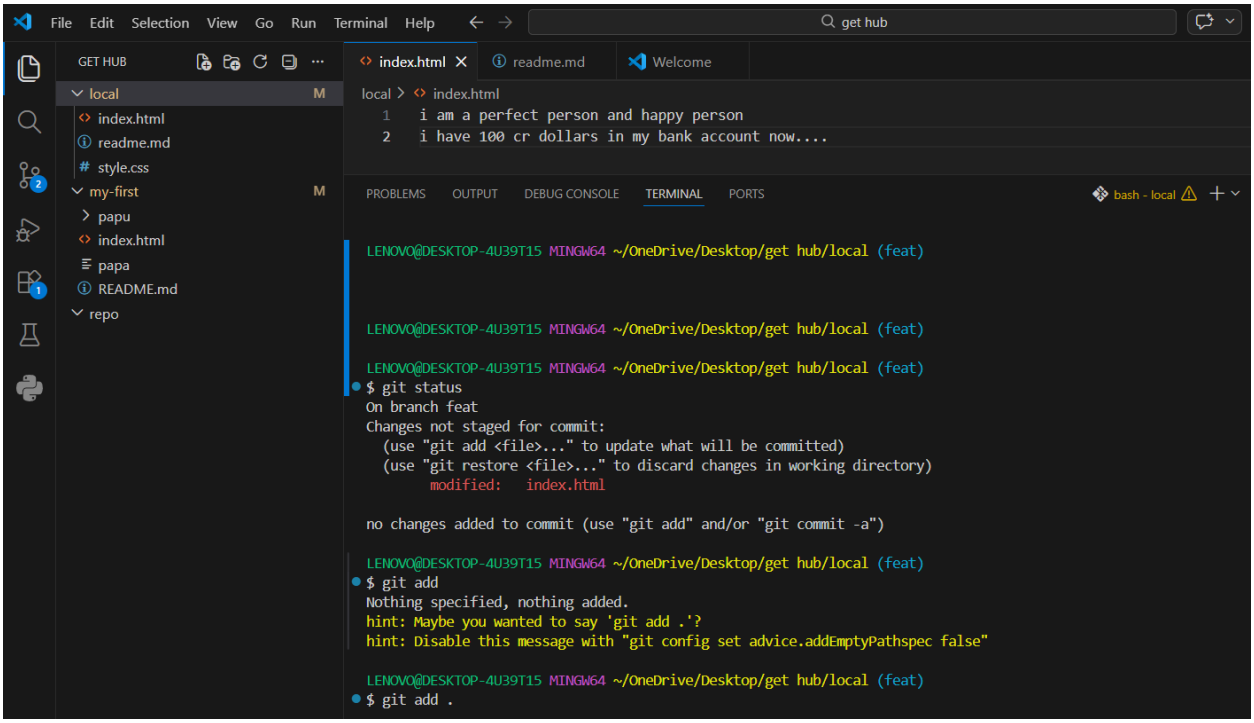
## git branch -d → Delete a branch

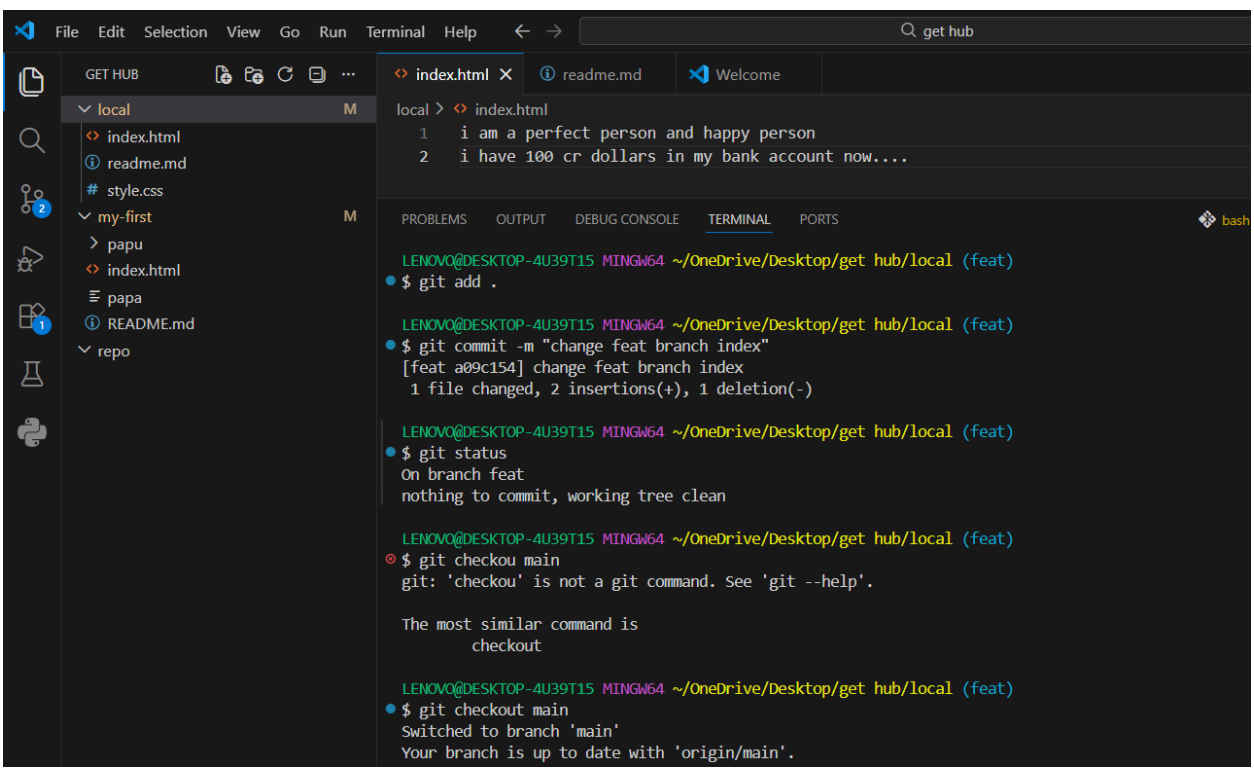


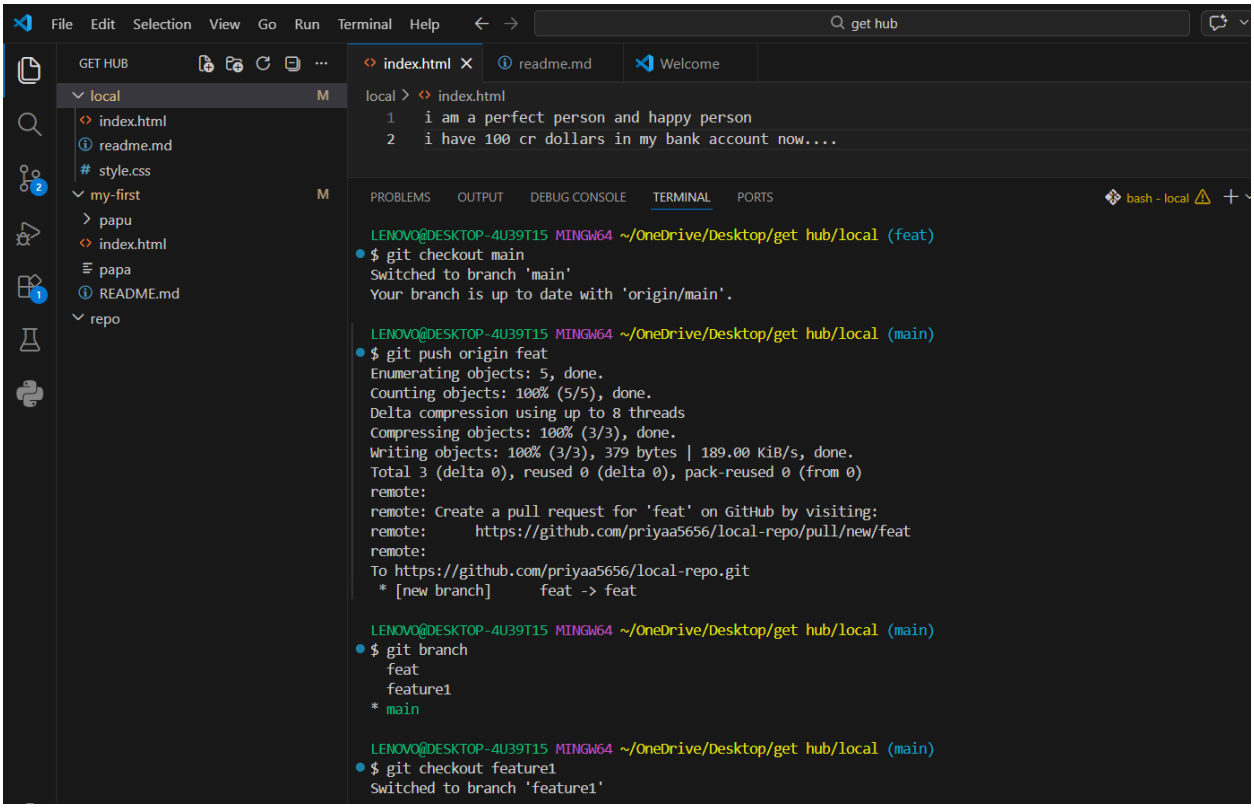



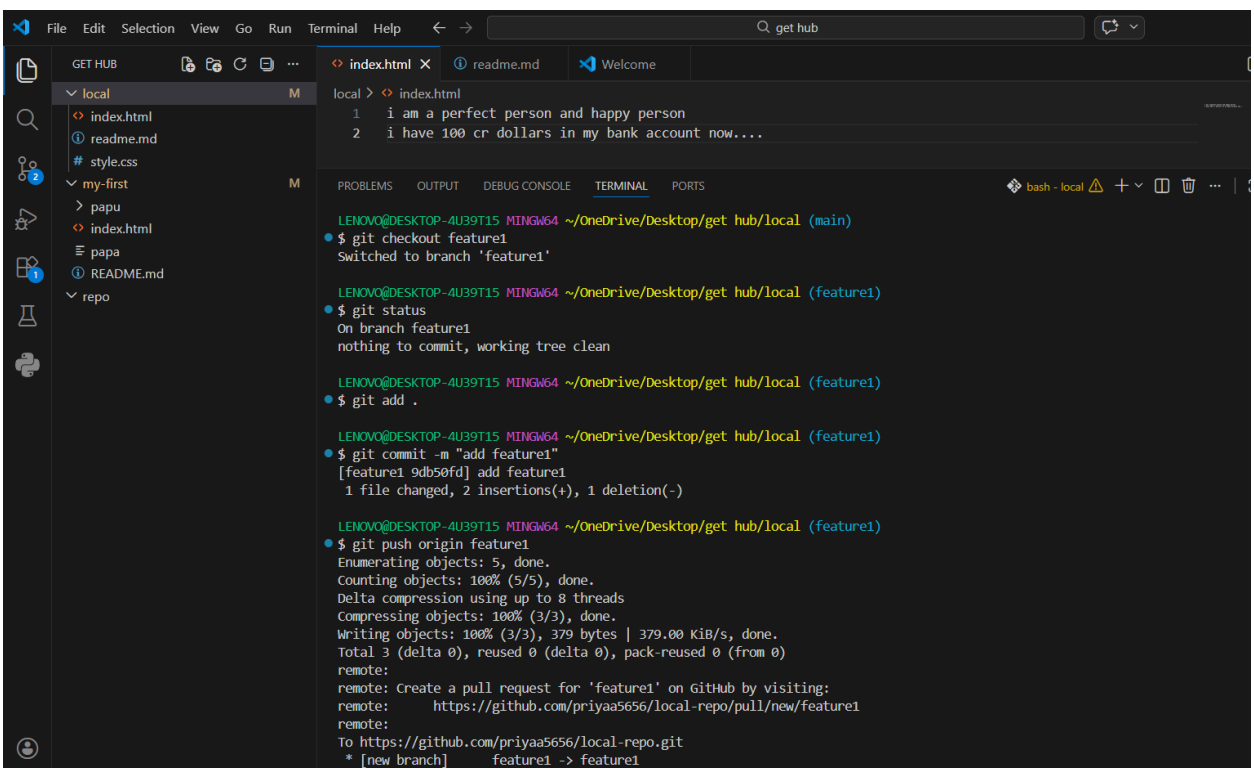



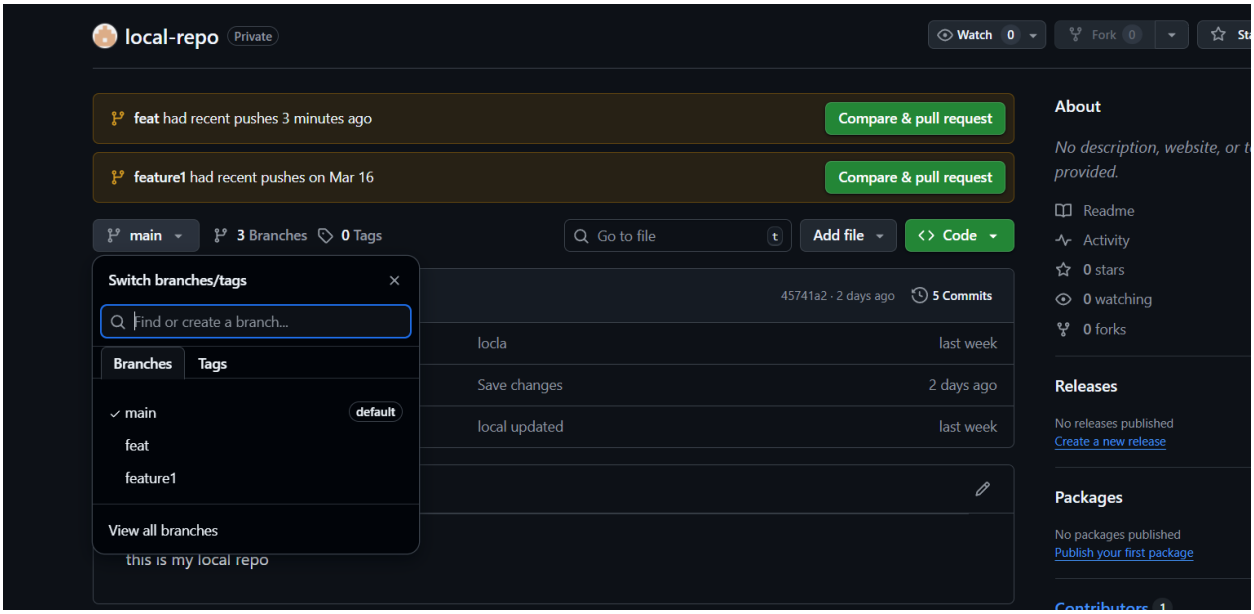
# Now if we want to change in any branch”:




File Edit Selection View Go Run Terminal Help
get hub
GET HUB
local
index.html
readme.md
style.css
my-first
papu
index.html
papa
README.md
repo
index.html
readme.md
Welcome
local > index.html
1 i am a perfect person and happy person
2 i have 100 cr dollars in my bank account now....
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - local
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (feat)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git push origin feat
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 379 bytes | 189.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feat' on GitHub by visiting:
remote: https://github.com/priyaa5656/local-repo/pull/new/feat
remote:
To https://github.com/priyaa5656/local-repo.git
* [new branch] feat -> feat
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git branch
feat
feature1
* main
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git checkout feature1
Switched to branch 'feature1'


File Edit Selection View Go Run Terminal Help
get hub
GET HUB
local
index.html
readme.md
style.css
my-first
papu
index.html
papa
README.md
repo
index.html
readme.md
Welcome
local > index.html
1 i am a perfect person and happy person
2 i have 100 cr dollars in my bank account now....
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - local
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git checkout feature1
Switched to branch 'feature1'
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (feature1)
$ git status
On branch feature1
nothing to commit, working tree clean
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (feature1)
$ git add .
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (feature1)
$ git commit -m "add feature1"
[feature1 9db50fd] add feature1
1 file changed, 2 insertions(+), 1 deletion(-)
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (feature1)
$ git push origin feature1
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 379 bytes | 379.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature1' on GitHub by visiting:
remote: https://github.com/priyaa5656/local-repo/pull/new/feature1
remote:
To https://github.com/priyaa5656/local-repo.git
* [new branch] feature1 -> feature1




# Merging code:

Now we want our main or feat branch is merging

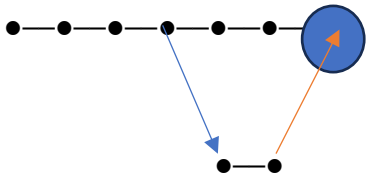
**Example**

**1 → 2 → 3 → 4 ↘**

**↘ ↘**

**5 → 6 →** 7 (merging my branches)

**or**

**main:**  (merging my branches)

**feature:**

## first way: through git or command line..

git diff <- branch name-> ( to compare commits, branches , files & more)

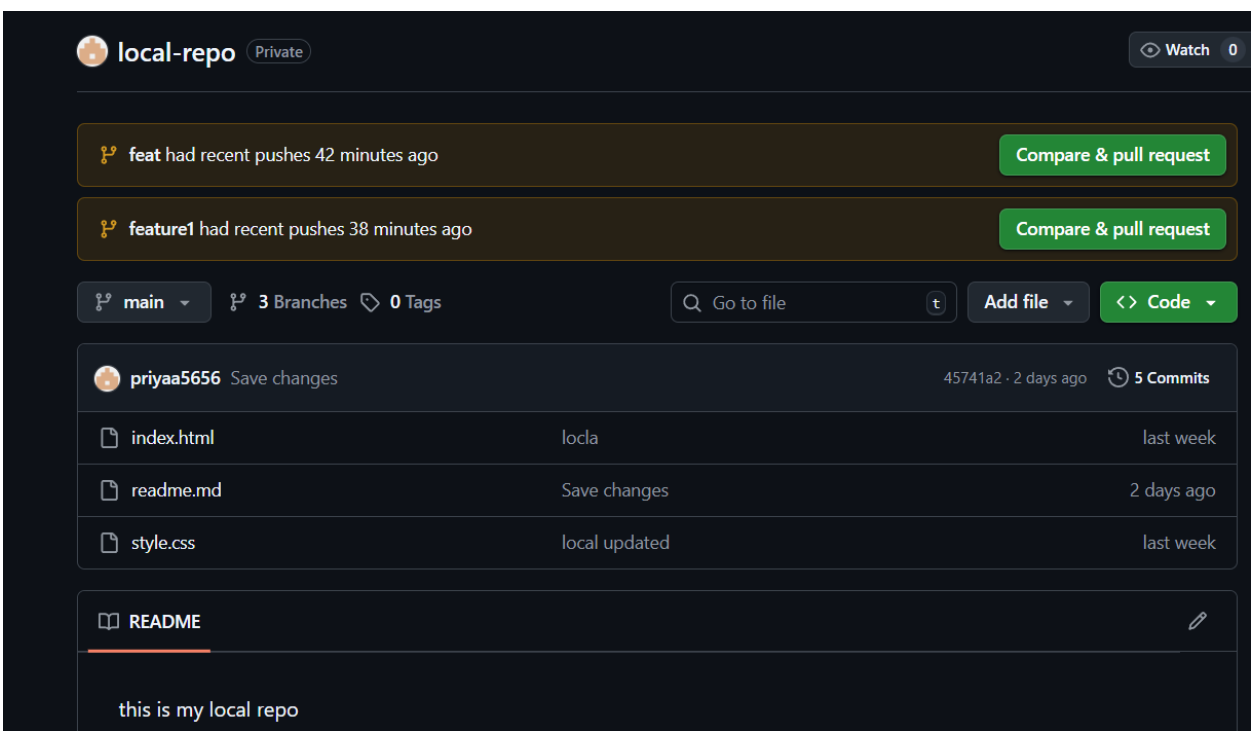
git merge <- branch name-> to merge 2 branches

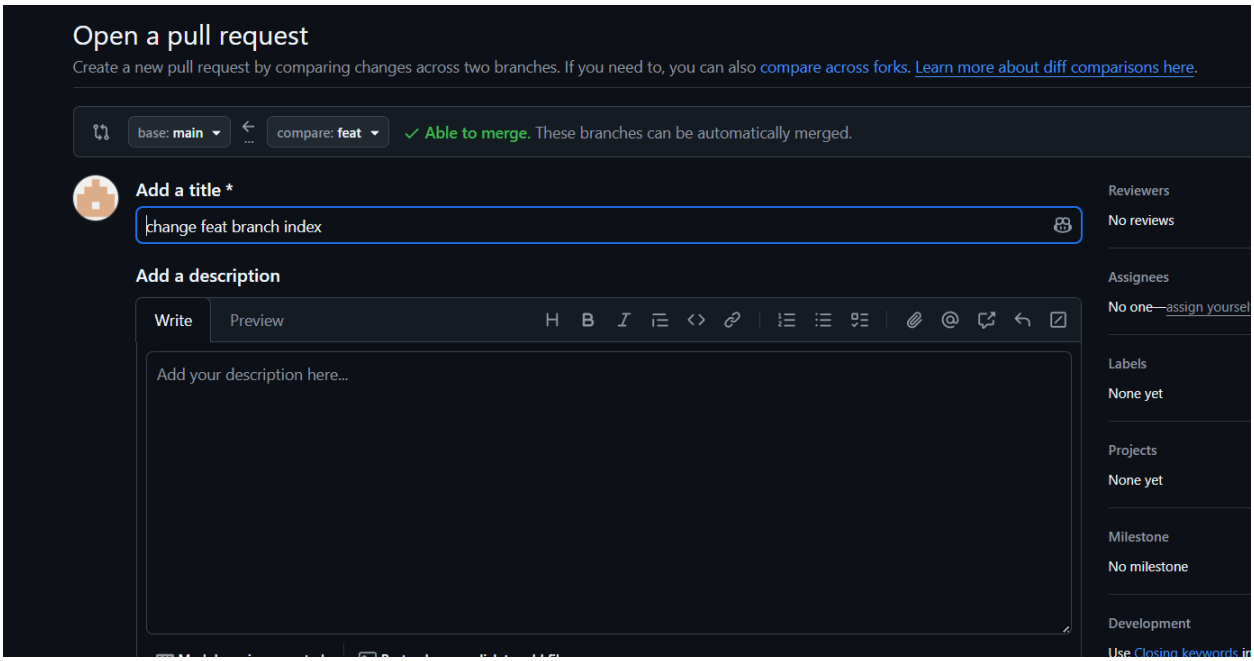
```
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (feature1)
$ git branch
  feat
* feature1
  main

LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (feature1)
$ git diff main
diff --git a/index.html b/index.html
index 799ffa9..8b2921c 100644
--- a/index.html
+++ b/index.html
@@ -1 +1,2 @@
-i am a perfect person and happy person
\ No newline at end of file
+i am a perfect person and happy person
+i have 100 cr dollars in my bank account now....
\ No newline at end of file
```

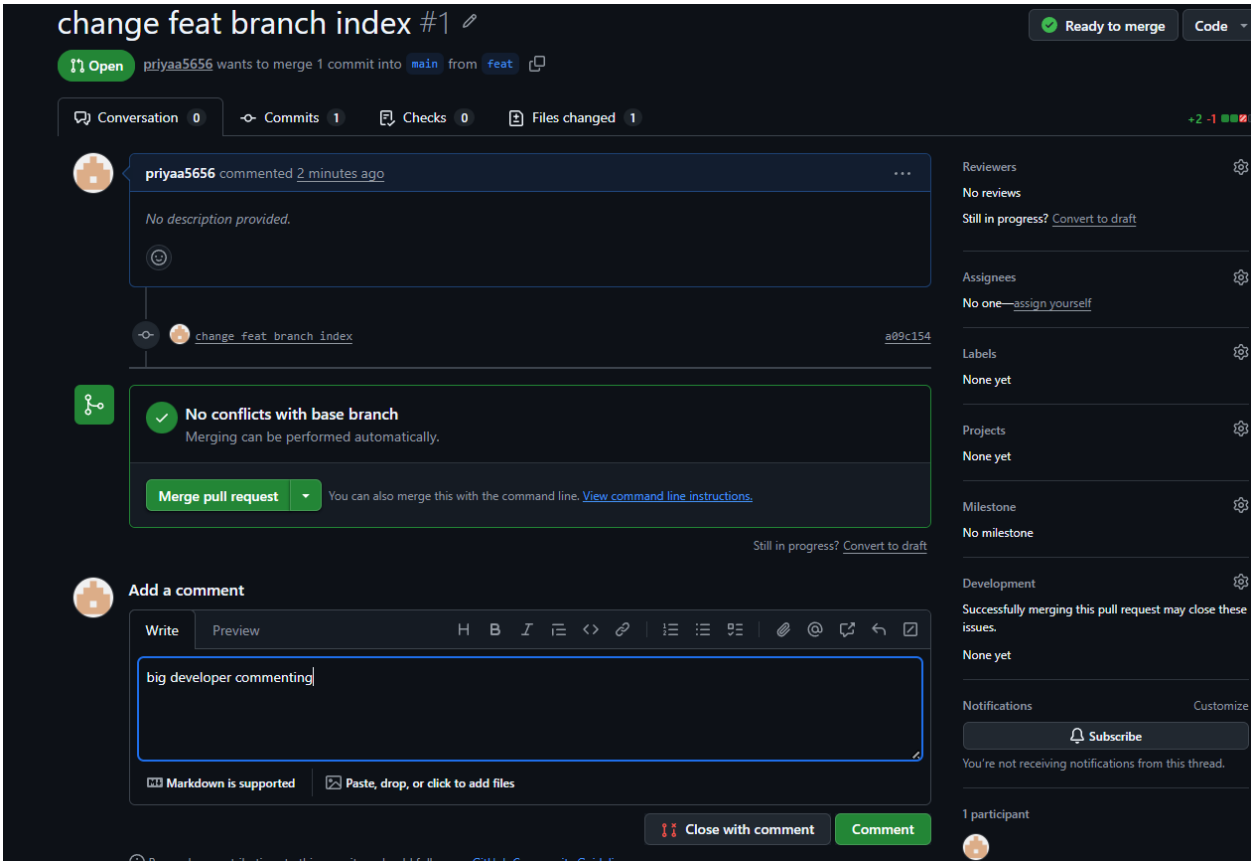
This full example is under /resolving merge conflicts.

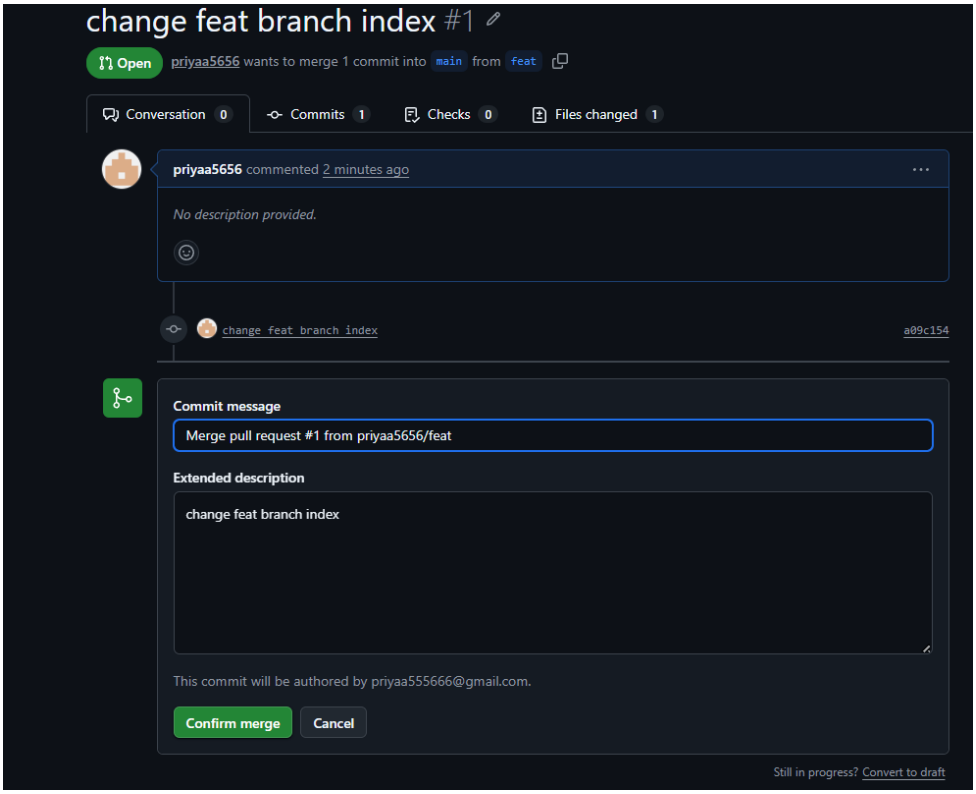
## 2nd way:

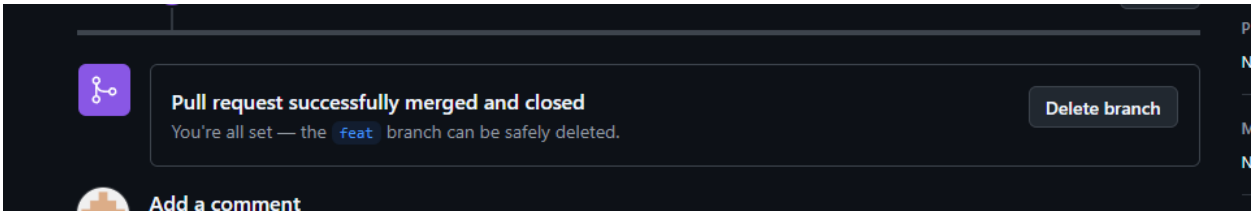
Create a PR means pull request  (it lets you tell others about changes you've pushed to a branch in a repository on github.

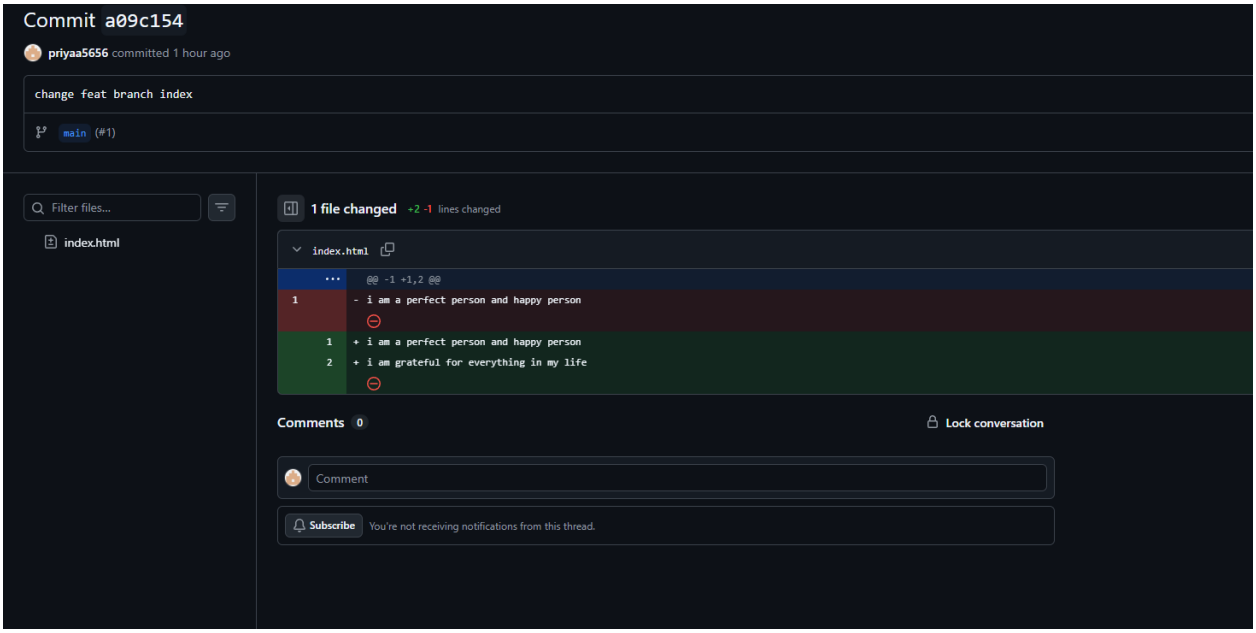



Open a pull request
Create a new pull request by comparing changes across two branches. If you need to, you can also compare across forks. Learn more about diff comparisons here.
base: main
compare: feat
Able to merge. These branches can be automatically merged.
Add a title *
change feat branch index
Add a description
Write
Preview
Add your description here...
Reviewers
No reviews
Assignees
No one—assign yourself
Labels
None yet
Projects
None yet
Milestone
No milestone
Development


change feat branch index #1
Ready to merge
Code
Open
priyaa5656 wants to merge 1 commit into main from feat
Conversation 0
Commits 1
Checks 0
Files changed 1
+2 -1
priyaa5656 commented 2 minutes ago
No description provided.
change feat branch index
a09c154
No conflicts with base branch
Merging can be performed automatically.
Merge pull request
You can also merge this with the command line. View command line instructions.
Still in progress? Convert to draft
Add a comment
Write
Preview
big developer commenting
Markdown is supported
Paste, drop, or click to add files
Close with comment
Comment
Reviewers
No reviews
Still in progress? Convert to draft
Assignees
No one—assign yourself
Labels
None yet
Projects
None yet
Milestone
No milestone
Development
Successfully merging this pull request may close these issues.
None yet
Notifications
Customize
Subscribe
You're not receiving notifications from this thread.
1 participant


change feat branch index #1
Open
priyaa5656 wants to merge 1 commit into main from feat
Conversation 0
Commits 1
Checks 0
Files changed 1
priyaa5656 commented 2 minutes ago
No description provided.
change feat branch index
a09c154
Commit message
Merge pull request #1 from priyaa5656/feat
Extended description
change feat branch index
This commit will be authored by priyaa555666@gmail.com.
Confirm merge
Cancel
Still in progress? Convert to draft

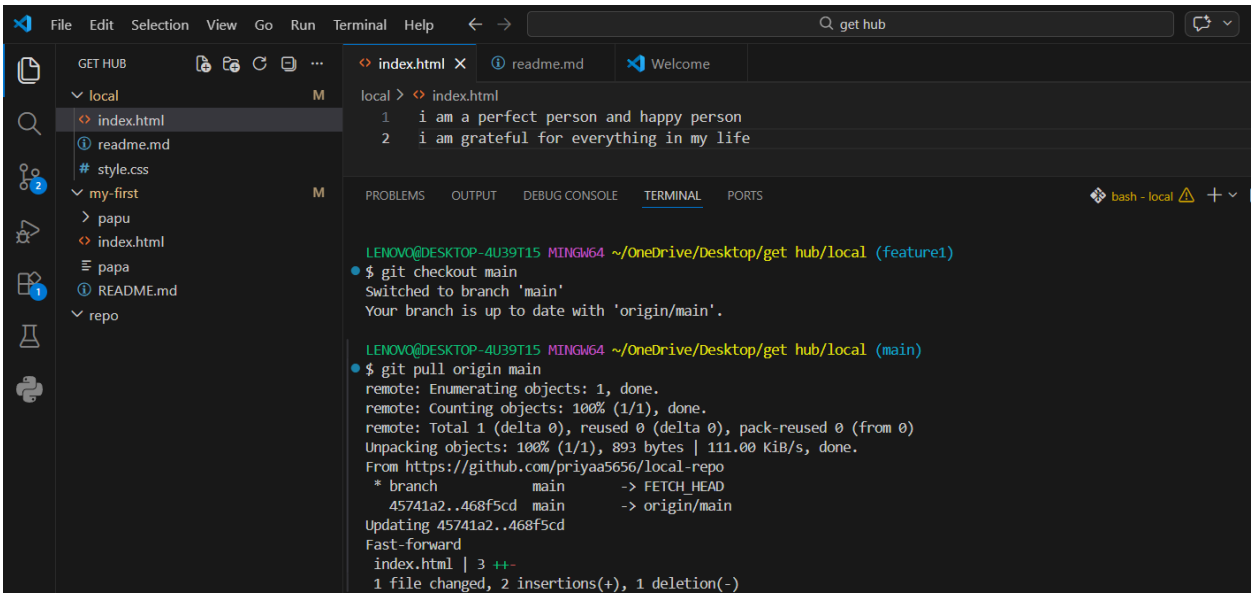

Pull request successfully merged and closed
You're all set — the feat branch can be safely deleted.
Delete branch
Add a comment


Commit a09c154
priyaa5656 committed 1 hour ago
change feat branch index
main (#1)
Filter files...
index.html
1 file changed +2 -1 lines changed
index.html
@@ -1 +1,2 @@
- i am a perfect person and happy person
+ i am a perfect person and happy person
+ i am grateful for everything in my life
Comments 0
Lock conversation
Comment
Subscribe
You're not receiving notifications from this thread.

Now changes are merge (feat changes merge in main). This method is change of local to git hub…

## Pull command:--- git pull origin main

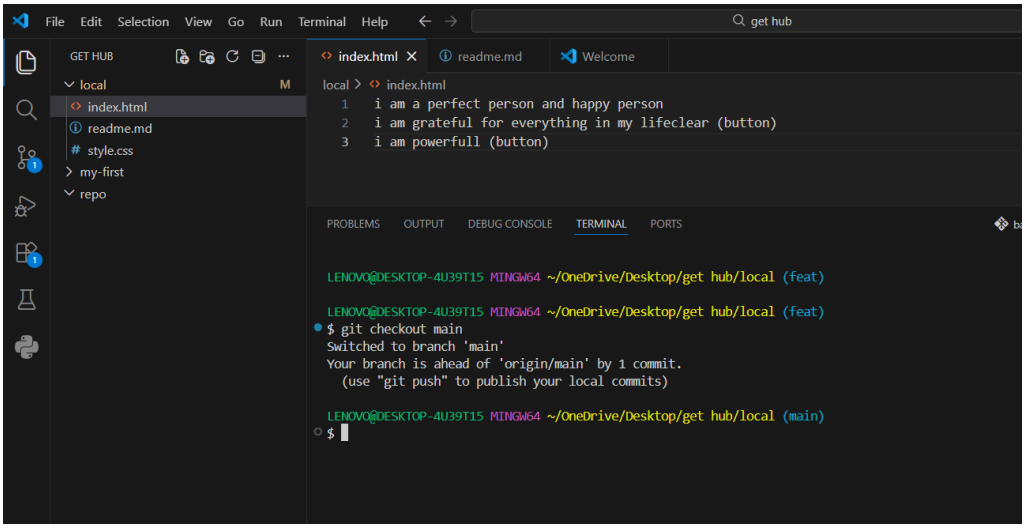
Used to fetch and download content  from a remote repo and immediately update the local repo to match that content. (Github to local)



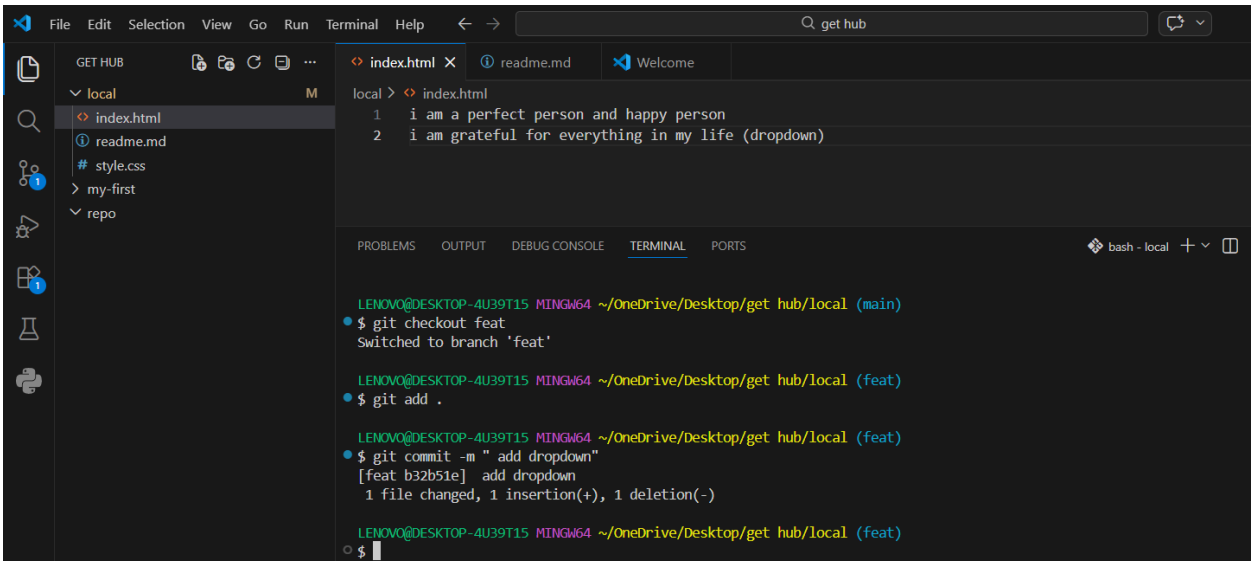

## Resolving merge conflicts→

An event that takes places when git is unable to automatically resolve differences in code between two commits.

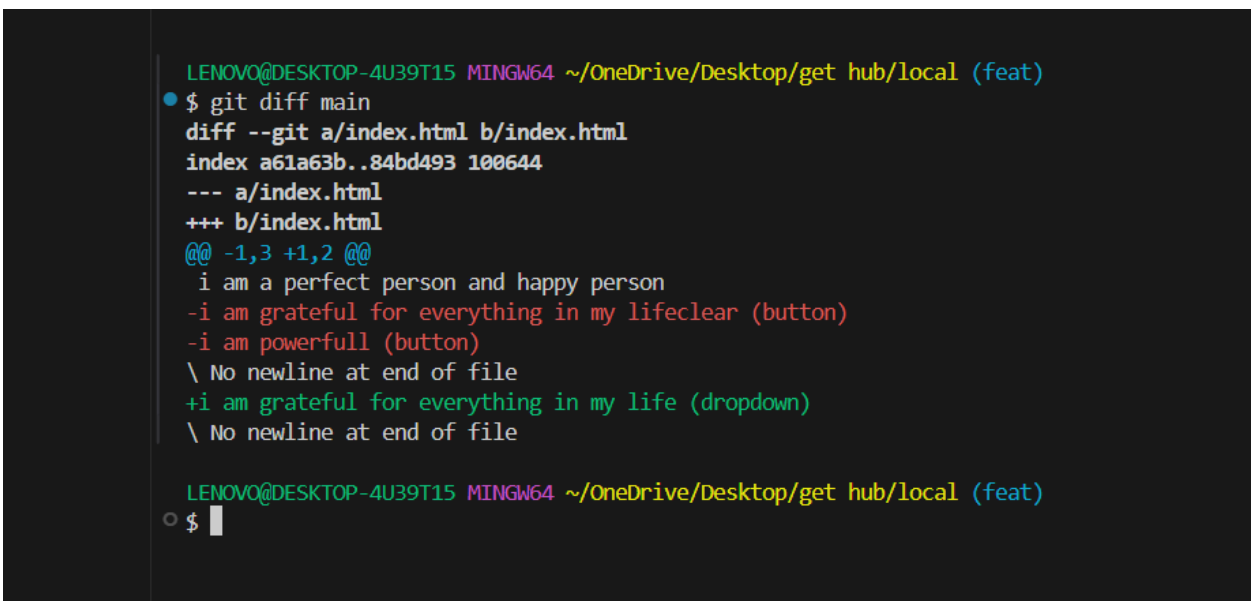
Example we change in main index and add then commit but we don’t see in feat branch



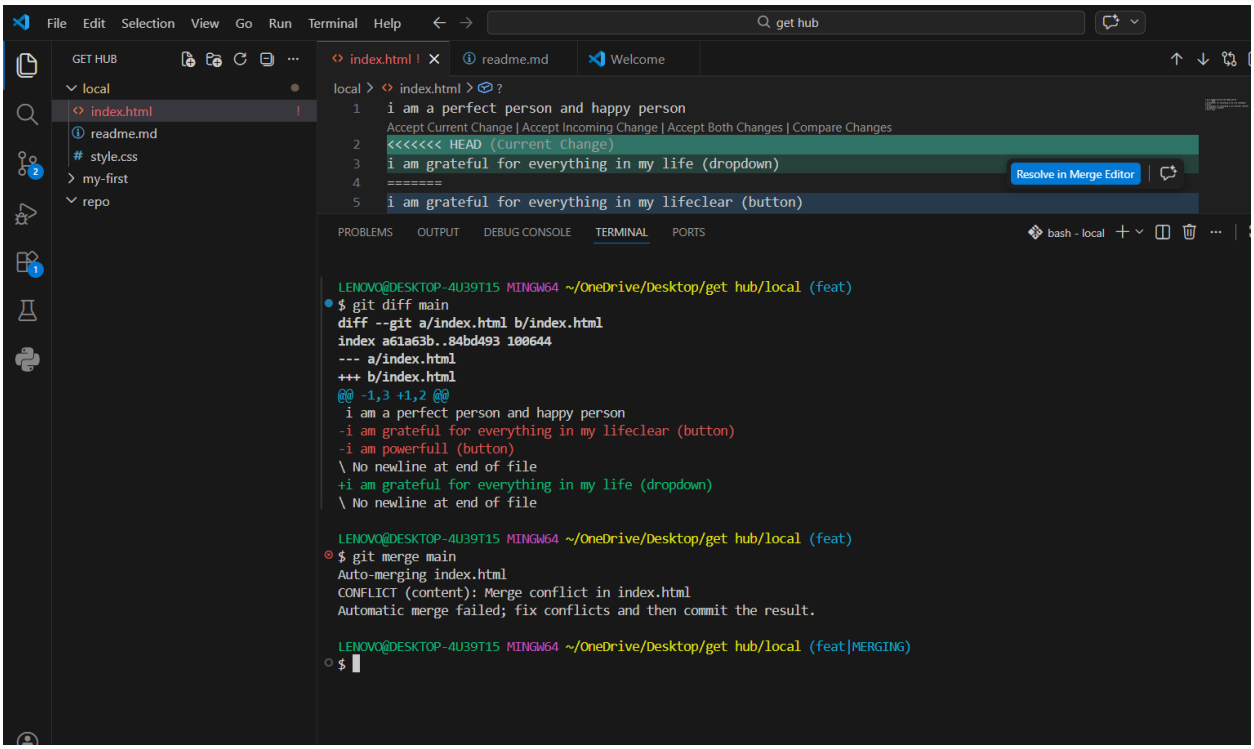
We add dropdown in feat branch




Now use diff.



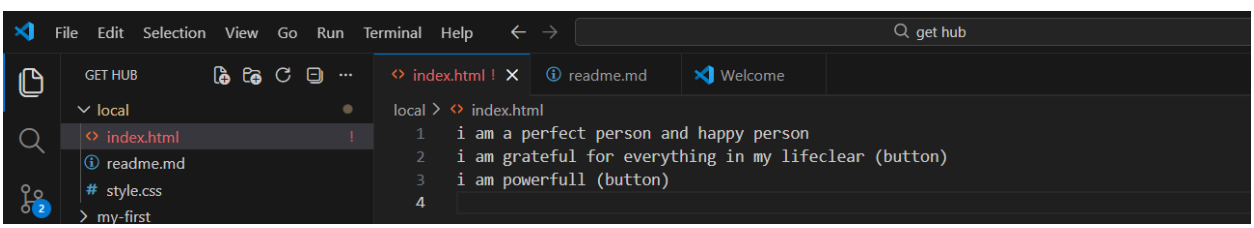

when we do changes on two different branches then want to merge the git is confused which branch is stay or remove. So in this case we told manually which branch change is stay or which is remove



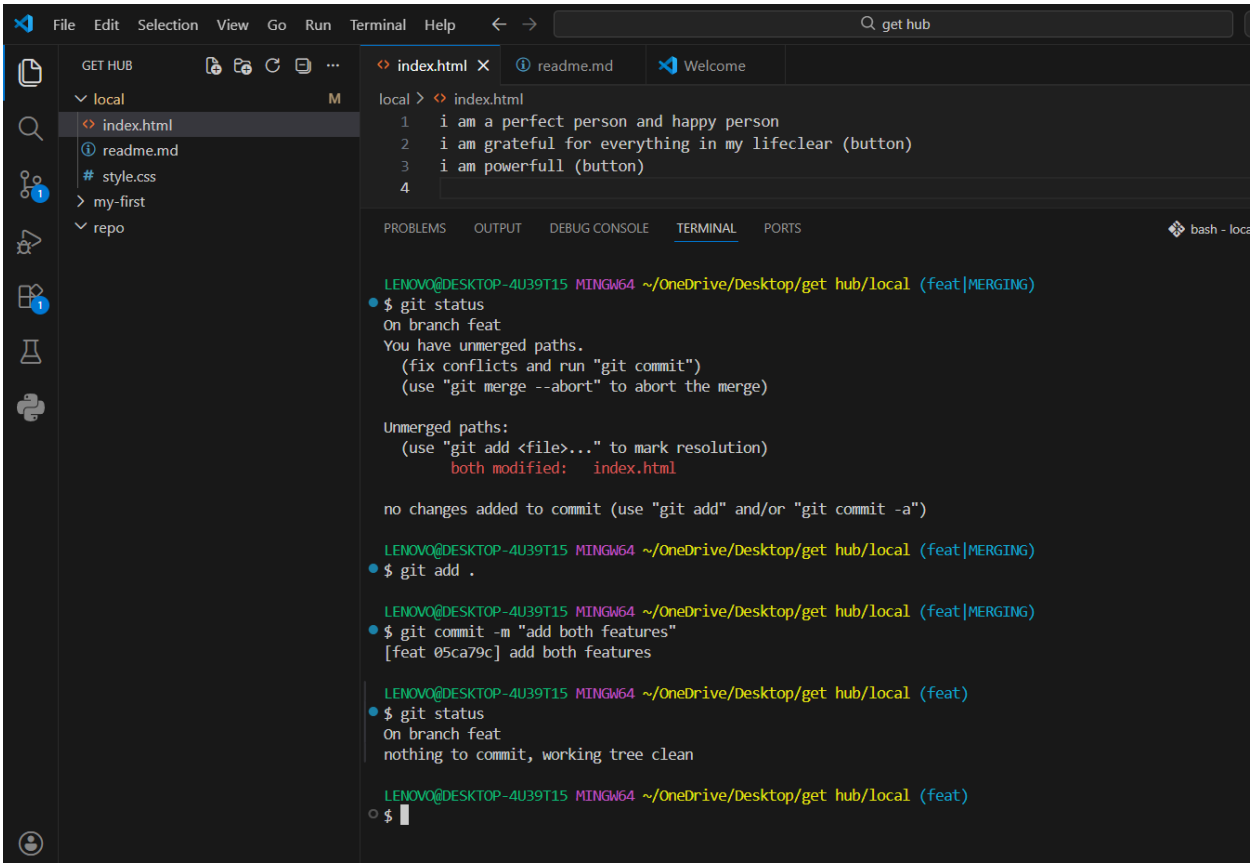

When we use git merge then we see error in the index.html so we chose one option in three. if you don't want do manually

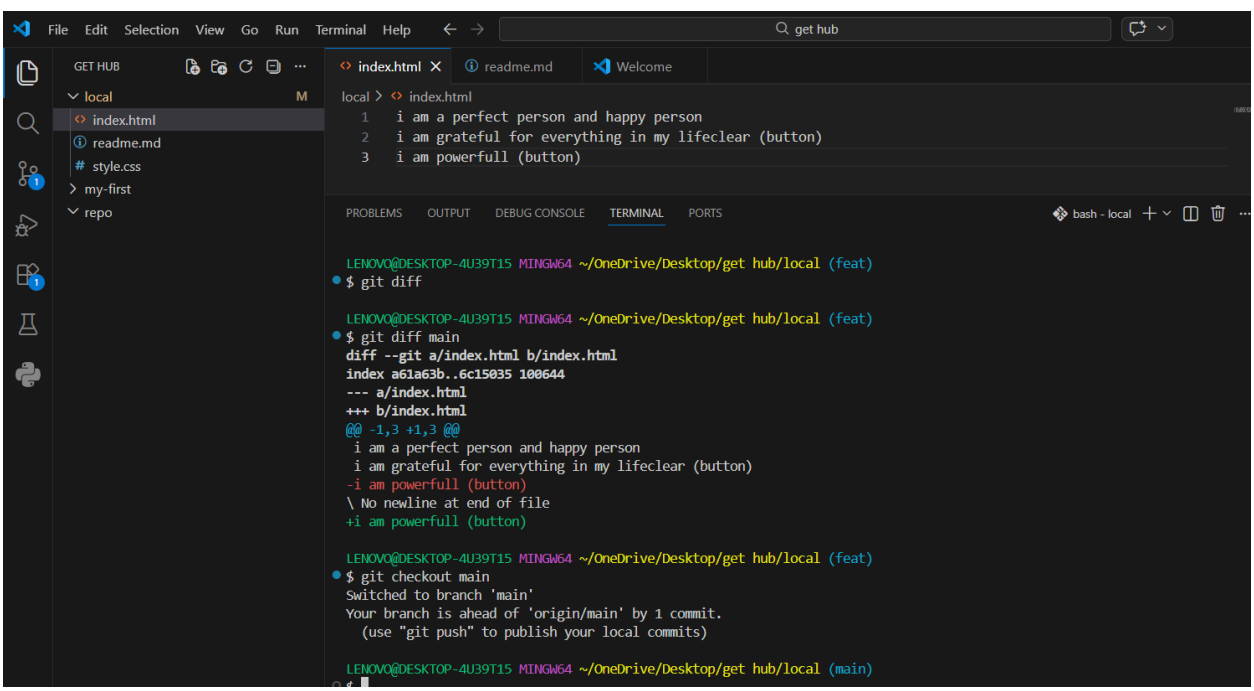
Accept current change/ Accept incoming change/ Accept both change/ compare change

Now we do manually we remove all the line like this ( <<<<< / -----)




And then add, commit




If our change is in same line is same so conflict is not coming otherwise you can see conflict.

Now if you want then git push

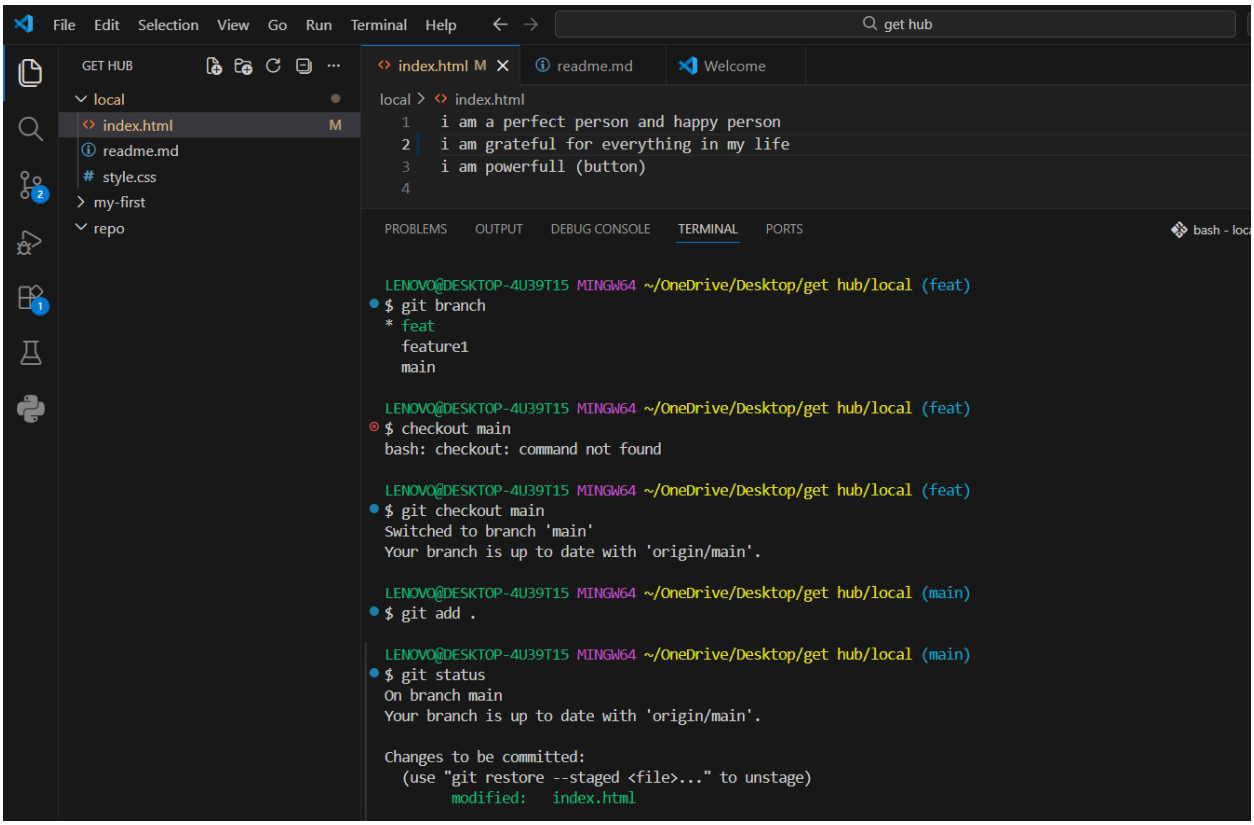
# Undoing changes:

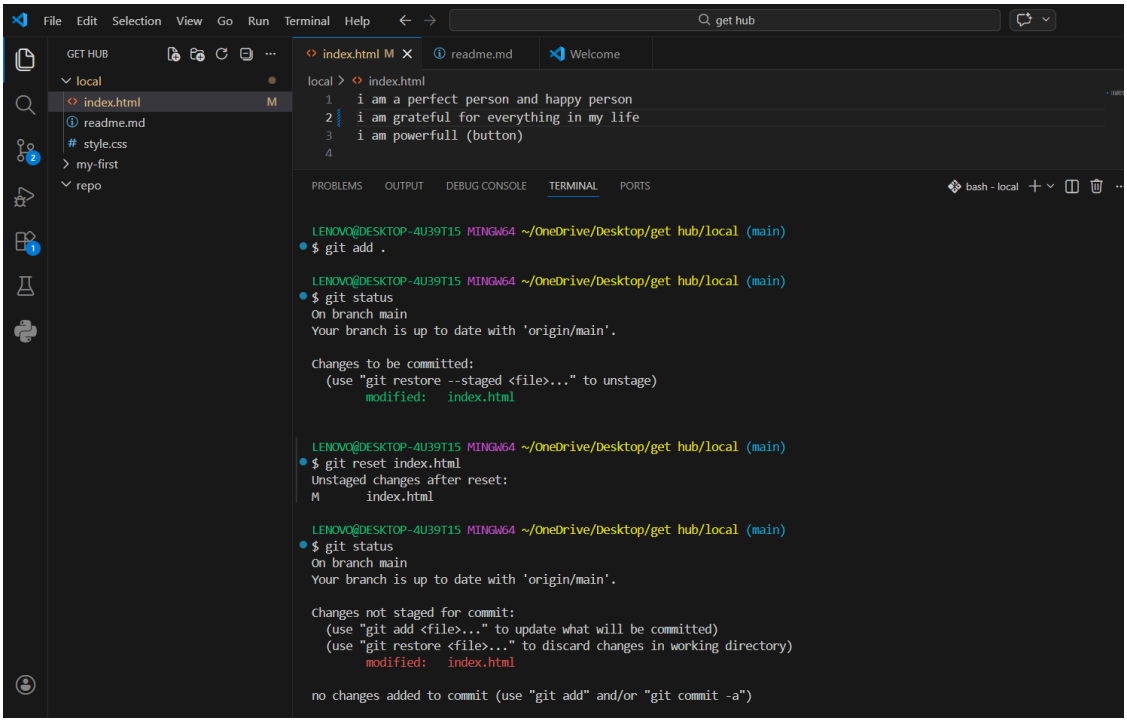
Some time we committed something which we should don't so how to undo this

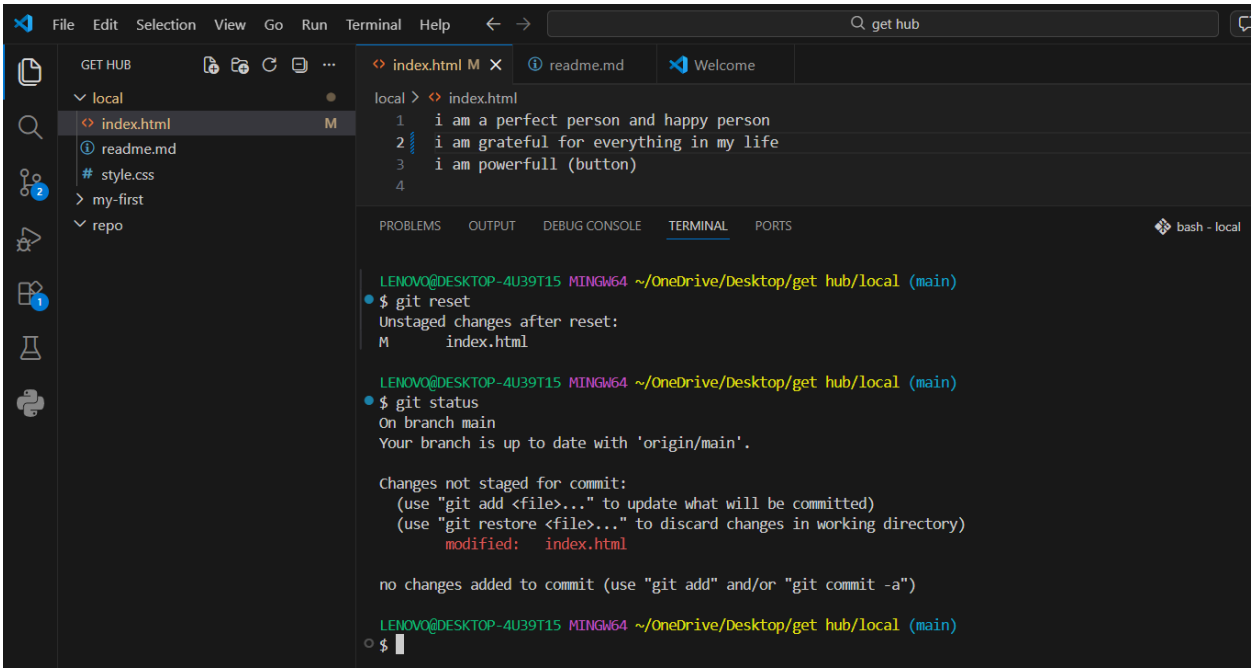
## Case1: staged changes (means add but not committed)

Git reset <-file name-> -→ for one or two files changes undo

Git reset --→ uncountable files changes undo




File Edit Selection View Go Run Terminal Help
get hub
GET HUB
local
index.html M
readme.md
style.css
my-first
repo
index.html M
readme.md
Welcome
local > index.html
i am a perfect person and happy person
i am grateful for everything in my life
i am powerfull (button)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - local
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git add .
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
modified: index.html
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git reset index.html
Unstaged changes after reset:
M index.html
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: index.html
no changes added to commit (use "git add" and/or "git commit -a")


File Edit Selection View Go Run Terminal Help
get hub
GET HUB
local
index.html M
readme.md
style.css
my-first
repo
index.html M
readme.md
Welcome
local > index.html
i am a perfect person and happy person
i am grateful for everything in my life
i am powerfull (button)
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
bash - local
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git reset
Unstaged changes after reset:
M index.html
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: index.html
no changes added to commit (use "git add" and/or "git commit -a")
LENOVO@DESKTOP-4U39T15 MINGW64 ~/OneDrive/Desktop/get hub/local (main)
$

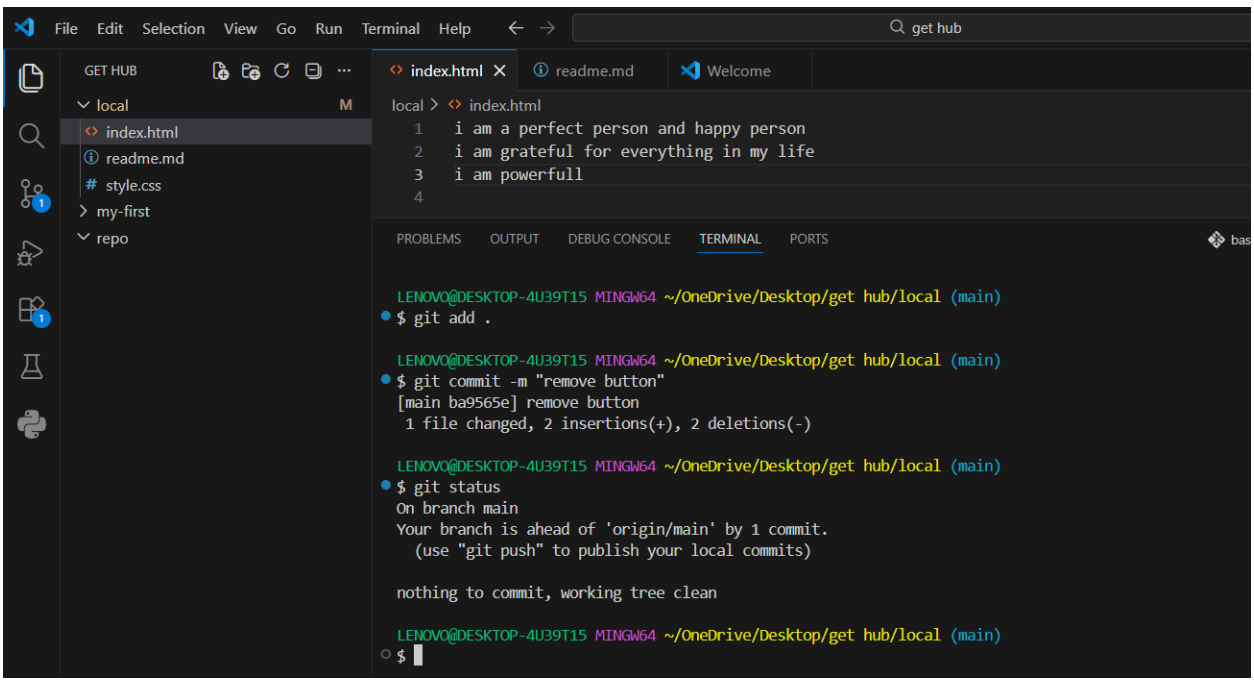
## Case 2: commited changes (for one commit) –( by mistake we commit some change)

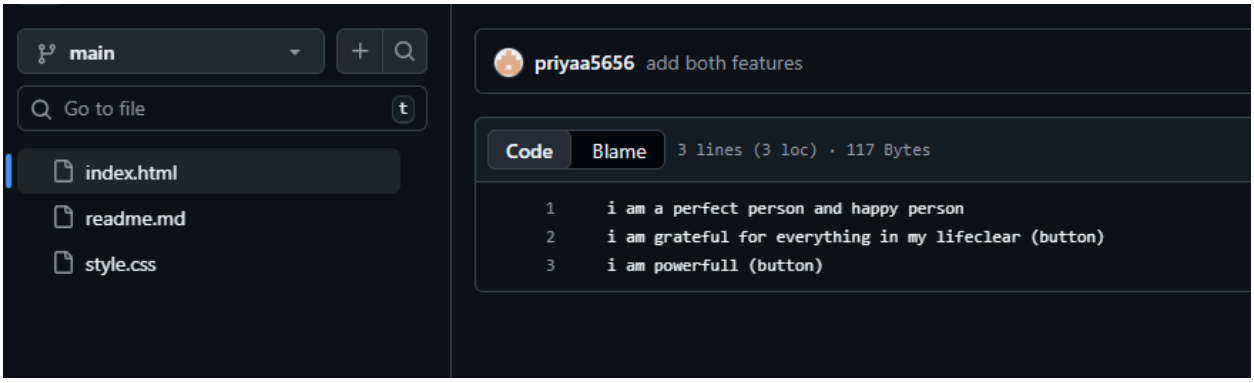
Git reset HEAD~1 ( HEAD~1 --- all the commit stored in ineternal git in this last commit is default name is HEAD .so that we said go 1 step back

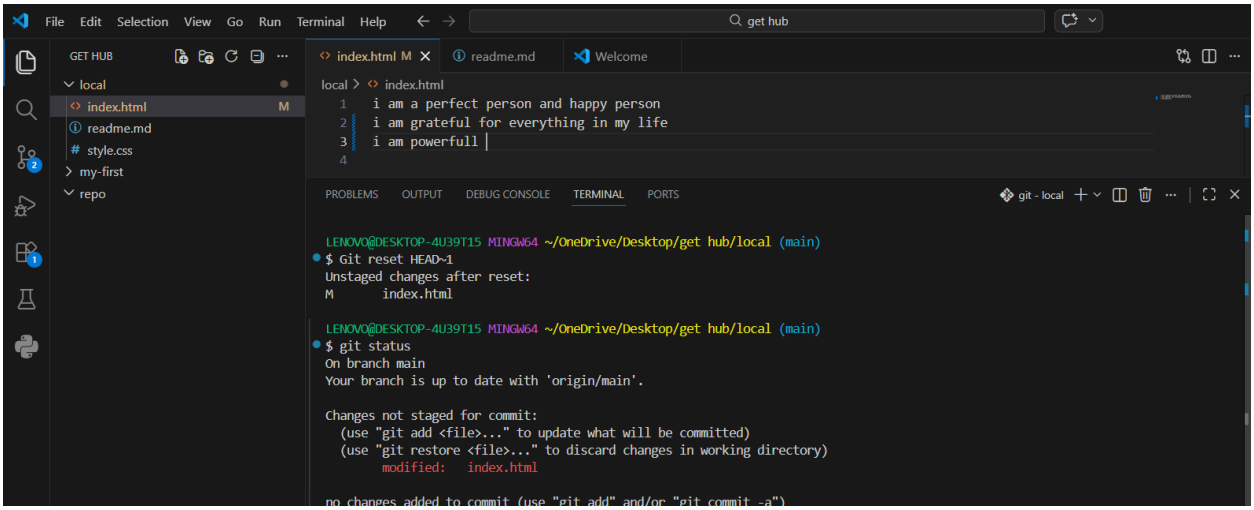
**1commit → 2 commit → 3 commit → 4 commit( Head)**

**So we want to go 3 commit use** Git reset HEAD~1 after

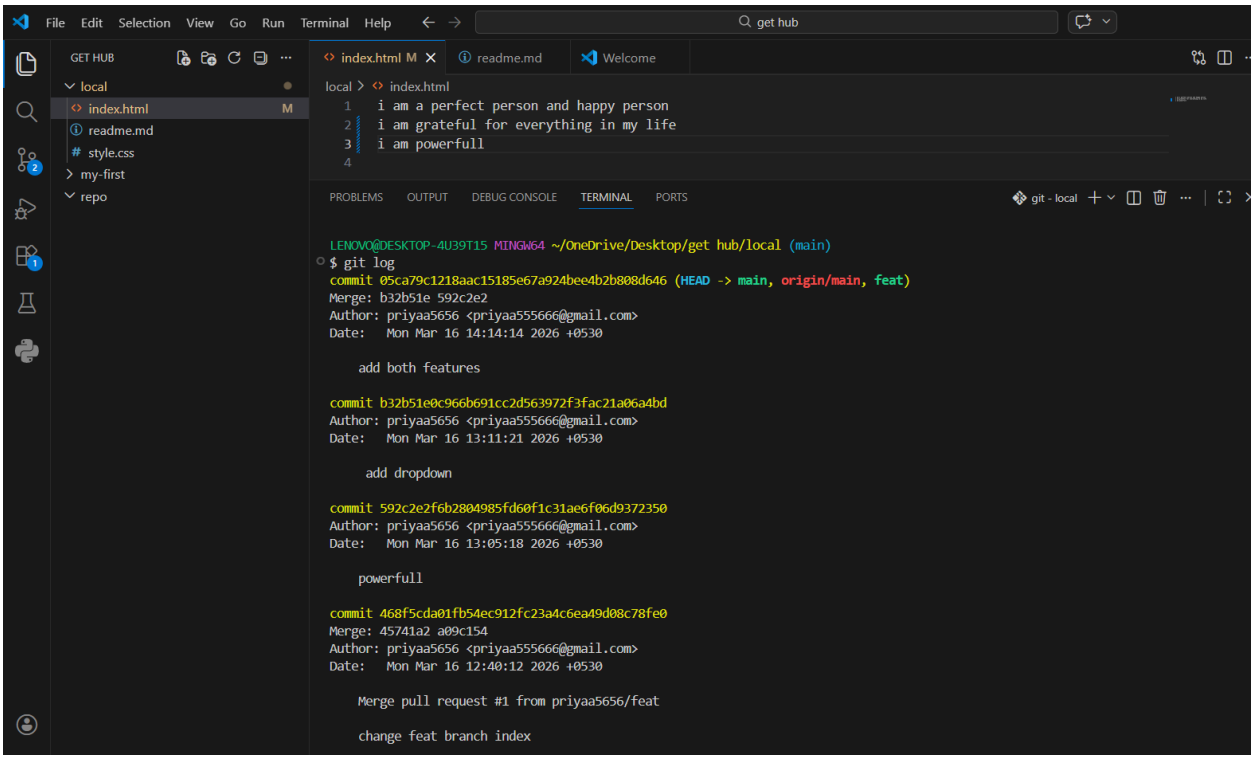
**1commit → 2 commit → 3 commit( Head)**






Use git log you see what commit you do some time ago



## Case3: commited changes (for many commits) →

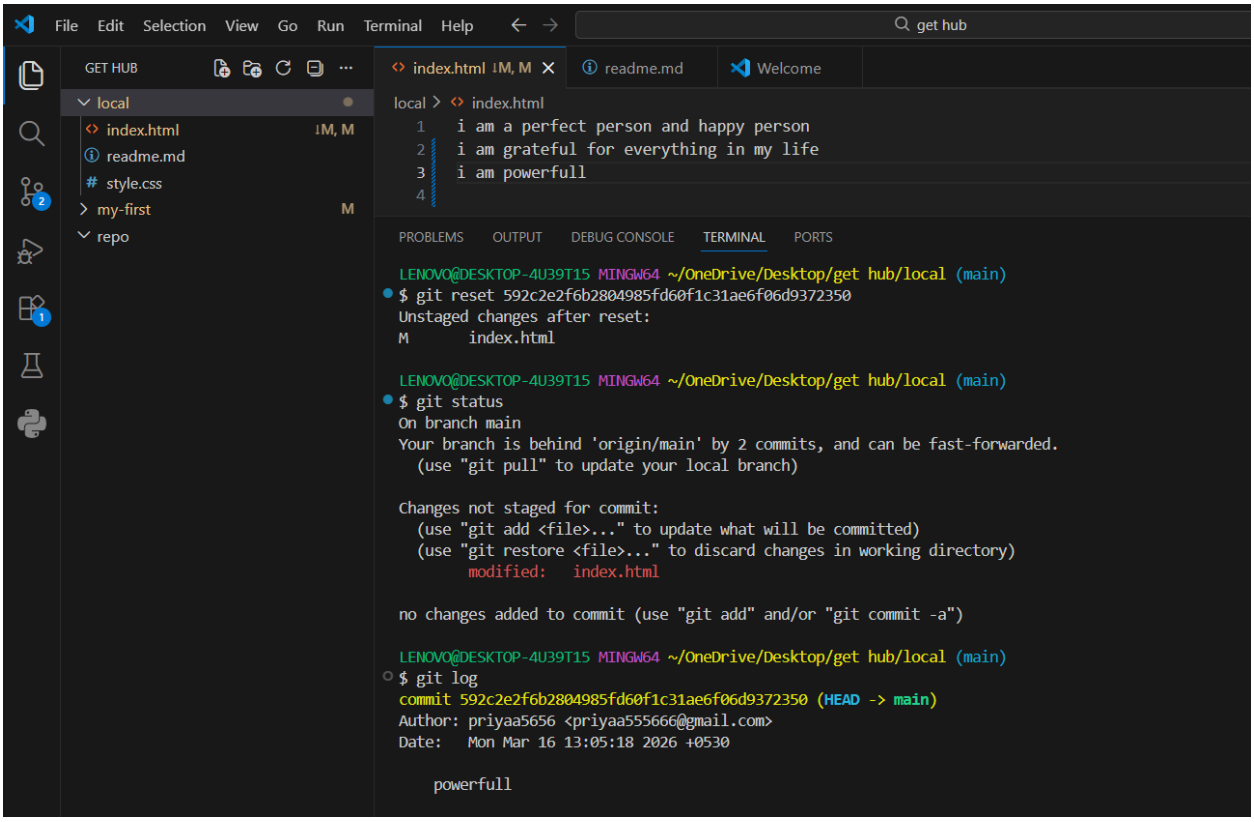
All commit has their unique has If we want to go on commit 1 so we copy commit 1 hash

**1commit → 2 commit → 3 commit → 4 commit( Head)**

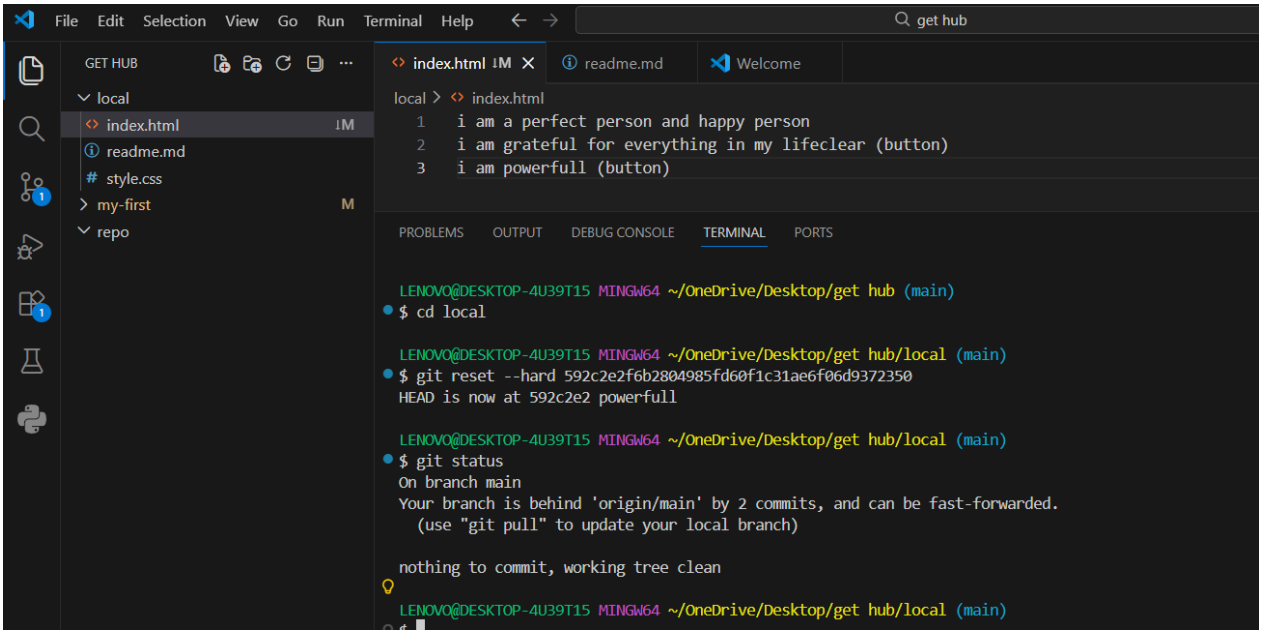
**So we want to go 3 commit use** Git reset hash after **1commit is ( Head) .**

Git reset <- commit hash ->

Git reset –hard <-commit hash ->h




Now we see powerful commit is coming on top. now our head is change




Fork :

A fork is a new repository that shares code and visibility settings with the original “upstream” repository. It is only a rough copy